KARAN SIR CLASSES · CFA LEARNING HUB

Platform **Technical Guide**

User workflows for candidates, staff and admins · backend and architecture mapping · APIs, database and UI mapping · scaling, performance and authentication.



1	2	3	4–5
UI workflows	Backend & architecture	APIs · DB · UI map	Scaling & authentication

Version 1.0
29 October 2026

Source of truth
Repository code (master @ 8b76491)

Audience
Product, engineering, operations, security

Contents

Part 0 · Overview 3
- 0.1 The platform at a glance 4
- 0.2 How to read this guide 5

Part 1 · UI workflows 6
- 1.1 Who uses the platform 7
- 1.2 Navigation maps 8
- 1.3 The candidate journey 9
- 1.4 Workflow C1 · Sign-in and sign-up 9
- 1.5 Workflow C2 · Under-18 candidates and guardian consent 11
- 1.6 Workflow C3 · Choosing, registering and paying 12
- 1.7 Workflow C4 · Learning 14
- 1.8 Workflow C5 · Tests and quizzes 16
- 1.9 Workflow C6 · Flashcards and spaced repetition 17
- 1.10 Workflow C7 · Search, bookmarks, orders and profile 18
- 1.11 Staff and admin workflows 19
- 1.12 Who can open which screen 23

Part 2 · Backend mapping 24
- 2.1 Technology stack 25
- 2.2 Technical architecture: a modular monolith 25
- 2.3 Request lifecycle 27
- 2.4 Domain events and the outbox 27
- 2.5 Deployment today: one container on Render 28
- 2.6 Target architecture: AWS Mumbai 29
- 2.7 Quality and delivery 30

Part 3 · APIs, database & UI mapping 31
- 3.1 API conventions 32
- 3.2 Endpoint inventory 33
- 3.3 Endpoint catalogue 33
- 3.4 Database design 38
- 3.5 UI → API → database mapping 43

Part 4 · Scaling & performance 46
- 4.1 Workload assumptions 47
- 4.2 Service-level objectives 48
- 4.3 How the code is built to scale 49
- 4.4 Known limits and bottlenecks 50
- 4.5 Scaling roadmap 51
- 4.6 Performance budgets and verification 51

Part 5 · Authentication & security 52
- 5.1 Defence in depth 53
- 5.2 Catalogue of authentication mechanisms 53
- 5.3 Candidate sign-in sequence 54
- 5.4 Sessions, cookies and refresh rotation 55
- 5.5 Authorisation 56
- 5.6 How credentials are stored 56
- 5.7 Integrity, audit and privacy controls 57
- 5.8 Threats and the controls that answer them 57
- 5.9 Gaps and hardening backlog 57
- 5.10 Appendix · source map 58

PART 0

Overview

What the platform is, how it is built, and how to read this guide.

0.1 The platform at a glance

CFA Learning Hub by Karan Sir Classes is a learning-management system for the CFA Level I exam. Candidates register and pay for a course, then study 56 learning modules (LMs) across 10 subjects with class videos, notes and PDFs. They test themselves with chapter quizzes, subject tests and custom quizzes, revise with spaced-repetition flashcards, and track progress. Staff author and publish content; admins run registrations, payments, people and compliance.

118 API endpoints	**37** database tables	**35** web pages (routes)	**5** roles · 32 permissions
12 backend modules	**56** learning modules	**32** automated tests	**15 min** access-token life

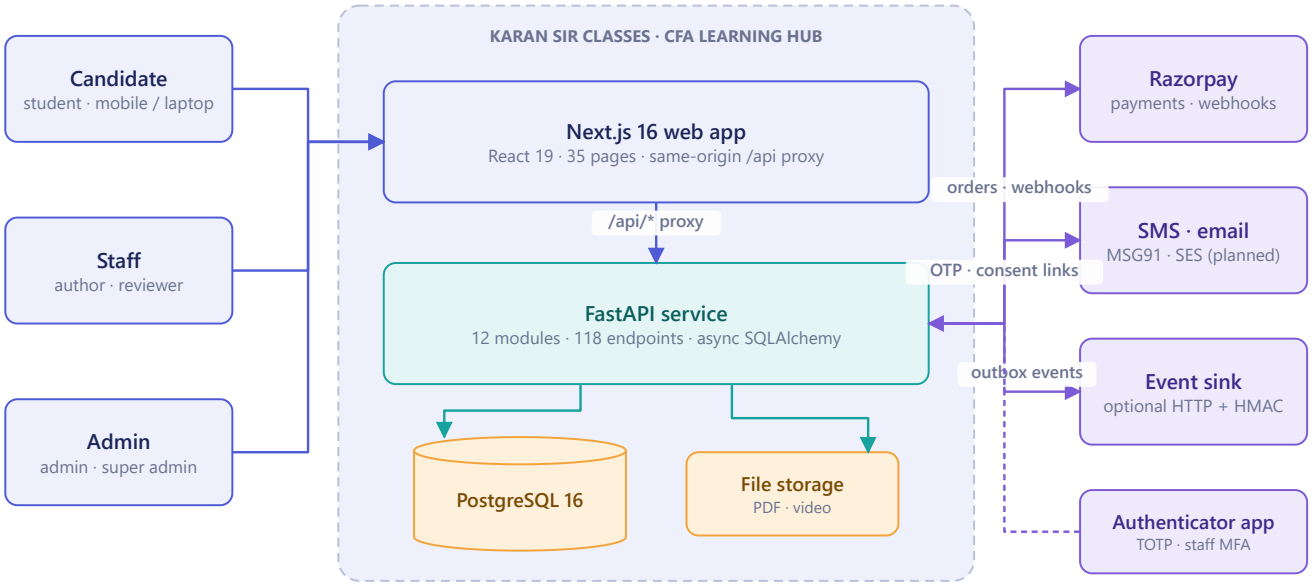


Figure 1. System context: three kinds of user, one web app, one API service, one database. Purple boxes are external services.

Technology in one table

Layer	Choice	Why it matters
Web	Next.js 16 · React 19 · TypeScript 5.9	Single origin. The browser only talks to the web host, which proxies `/api/*` to FastAPI. No CORS, simple cookies.
API	FastAPI · Pydantic 2 · Python 3.12	Typed request/response contracts, async I/O, one process per container worker.
Data	PostgreSQL 16 · SQLAlchemy 2 (async) · Alembic	Relational integrity (58 foreign keys), JSONB for flexible content, versioned migrations.
Auth	OTP · password + TOTP · JWT cookies · Argon2id	Mobile-first sign-in for candidates; MFA-enforced staff login.

Layer	Choice	Why it matters
Payments	Razorpay (live) · built-in mock gateway (demo)	Webhook-driven, idempotent, amount-verified.
Hosting	Docker image on Render (current) · AWS ap-south-1 (target)	One container runs both processes today; stateless tiers scale out later.

How this guide was produced
Every number here is read from the repository: endpoints from the router source, tables from the SQLAlchemy models, permissions from `rbac.py`, token lifetimes from `config.py`. Screenshots are real captures of the running application. Where a figure is a *target* or *model* (for example capacity), it is labelled as such.

0.2 How to read this guide

You are…	Start with	Then
Product owner / trainer	Part 1 · UI workflows	Part 5 for what protects candidate data
Backend / frontend engineer	Part 2 · Backend mapping	Part 3 · API, database and UI tables
DevOps / SRE	Part 2 §2.5–2.6 (deployment)	Part 4 · Scaling and performance
Security / compliance reviewer	Part 5 · Authentication	Part 1 §1.11 (privacy flows)

PART 1

UI workflows

How candidates, staff and admins move through the product, screen by screen.

1.1 Who uses the platform

Three audiences share one application. The menu a person sees is built from the permissions their roles carry, so a candidate never sees the staff console and an author never sees refunds.

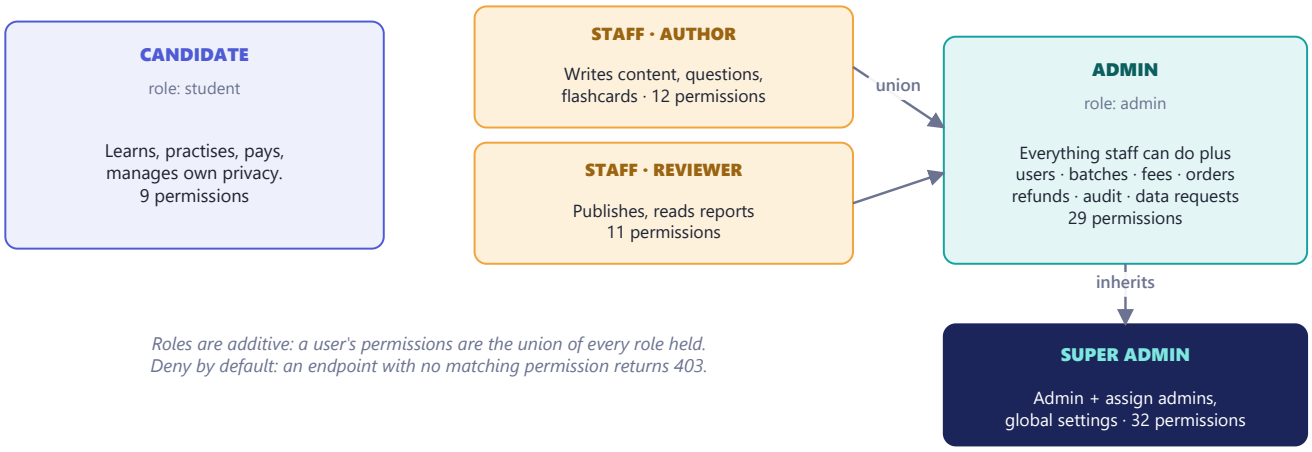


Figure 2. Roles and how they combine. Permission counts are taken from `core/rbac.py`.

Audience	Roles	Primary goals	Home screen
Candidate	student	Pass CFA Level I: learn, practise, revise, track progress. Pay and get receipts. Control personal data.	`/dashboard`
Staff	author, reviewer	Author: write notes, upload videos/PDFs, write questions and flashcards. Reviewer: publish content and read progress reports.	`/admin`
Admin	admin, super_admin	Run the business: courses and fees, registrations, orders and refunds, users and batches, privacy requests, audit trail. Super admin also assigns admins.	`/admin`

1.2 Navigation maps

Candidate application

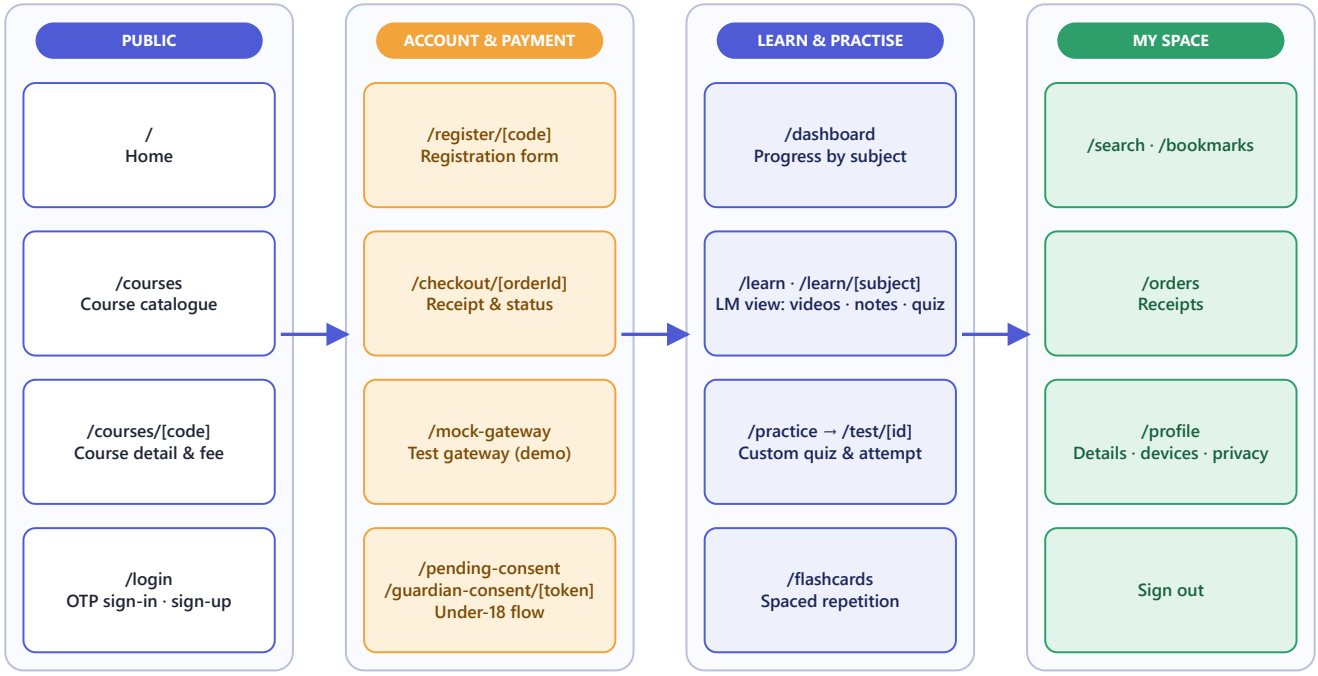


Figure 3. Candidate site map: 4 zones, 20 pages. Public pages need no account; everything in the right two zones needs sign-in.

Staff console



Figure 4. Staff console: 12 menu items grouped by job. Each menu item appears only when the user holds the permission shown beneath its group.

1.3 The candidate journey

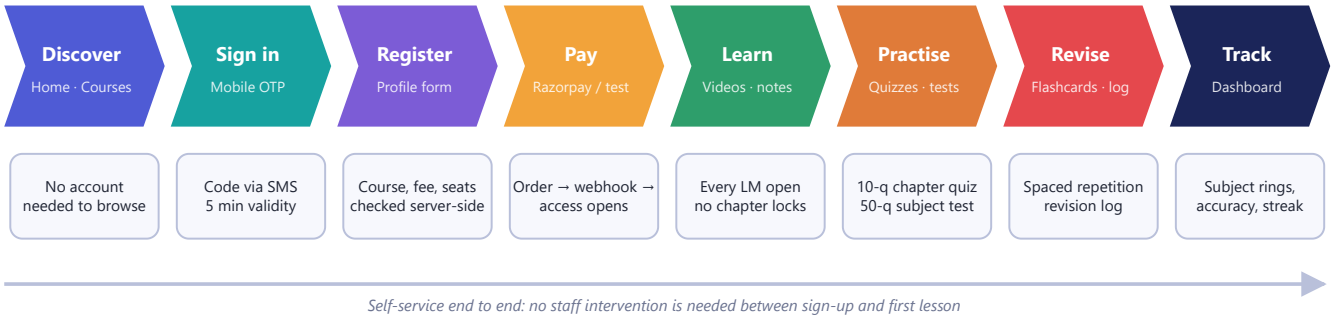


Figure 5. End-to-end candidate journey from discovery to progress tracking.

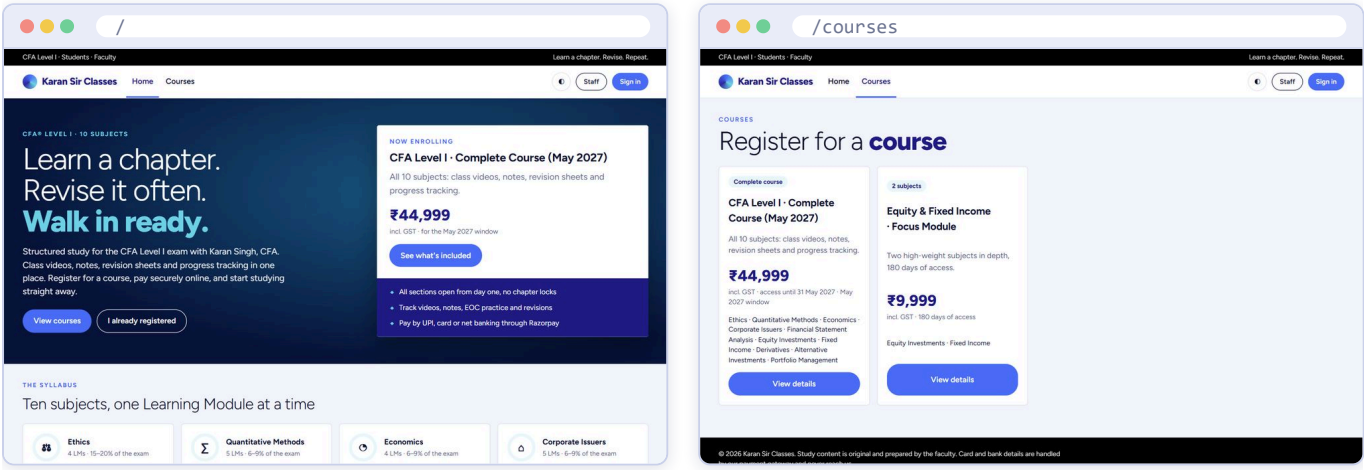


Figure 6. Public entry: the home page and the course catalogue (live capture).

1.4 Workflow C1 · Sign-in and sign-up

Candidates never create a password. A one-time code sent to their mobile proves ownership, and the same screen handles both existing and new accounts. Email OTP exists in the API for later use.

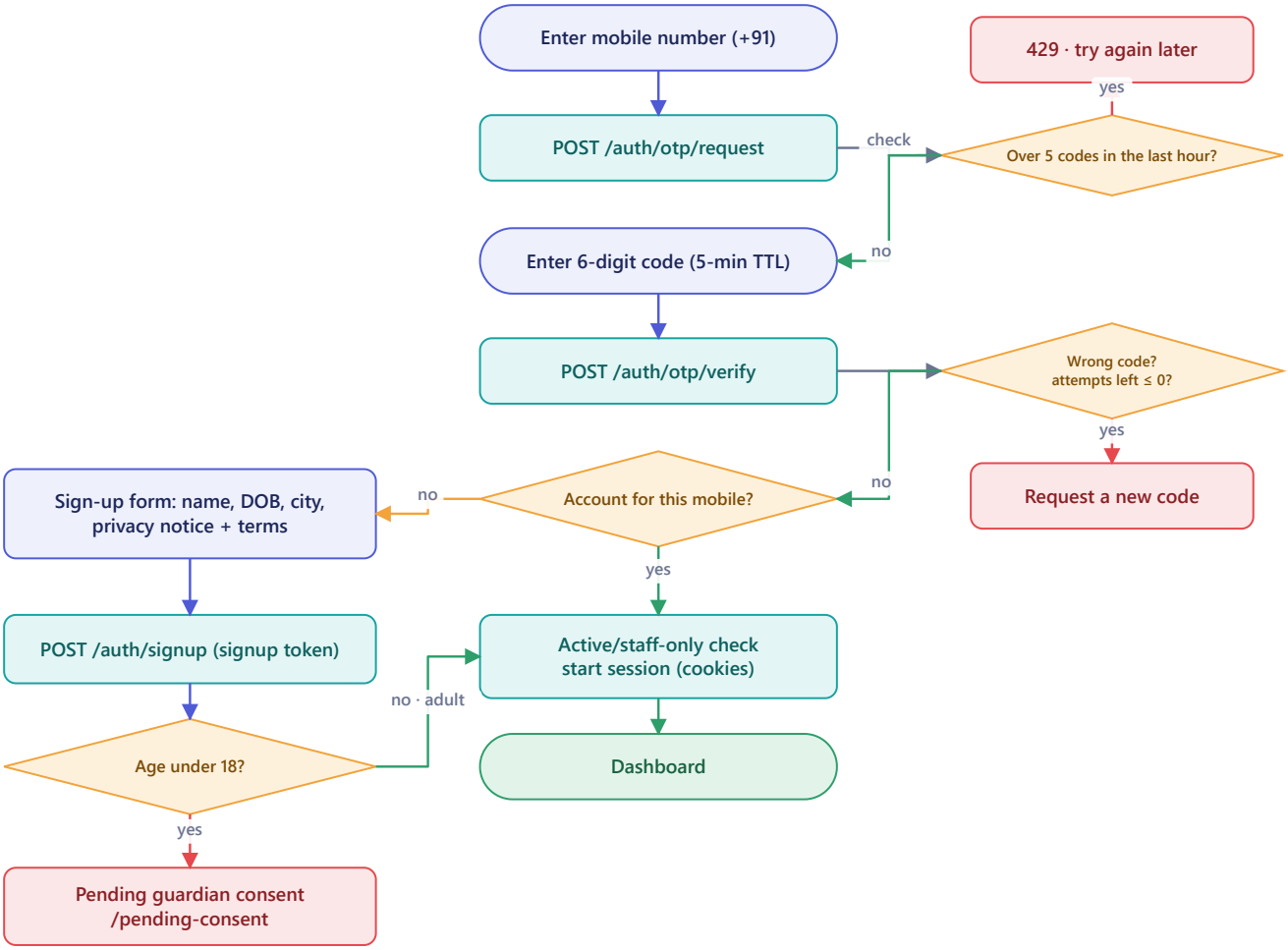


Figure 7. Sign-in decision flow. Diamonds are server decisions; red boxes are error states the UI explains in plain language.

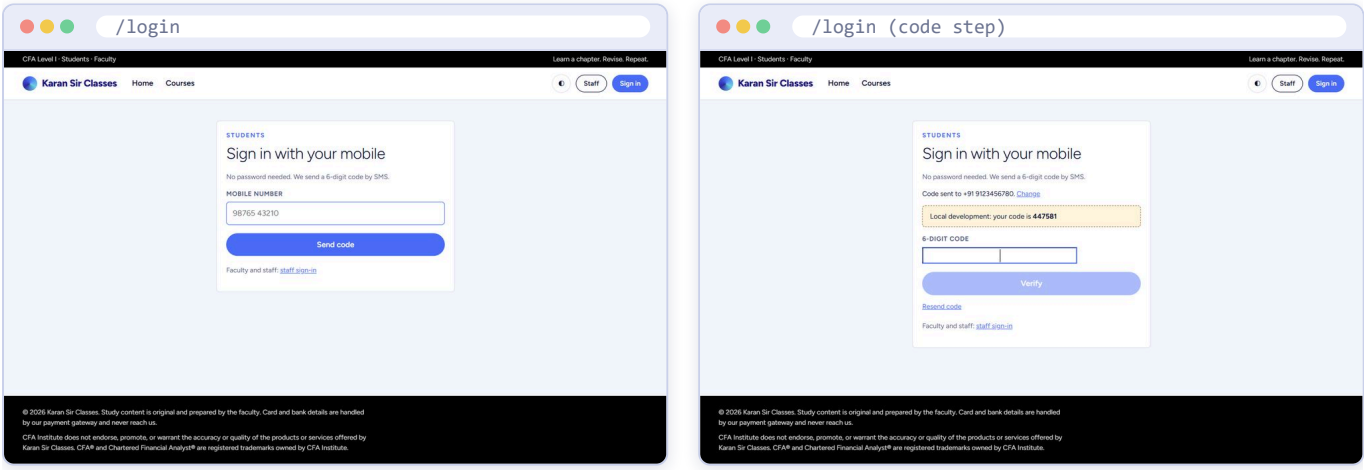


Figure 8. Step 1: mobile number. Step 2: six-digit code. Local and demo environments echo the code on screen; production never does.

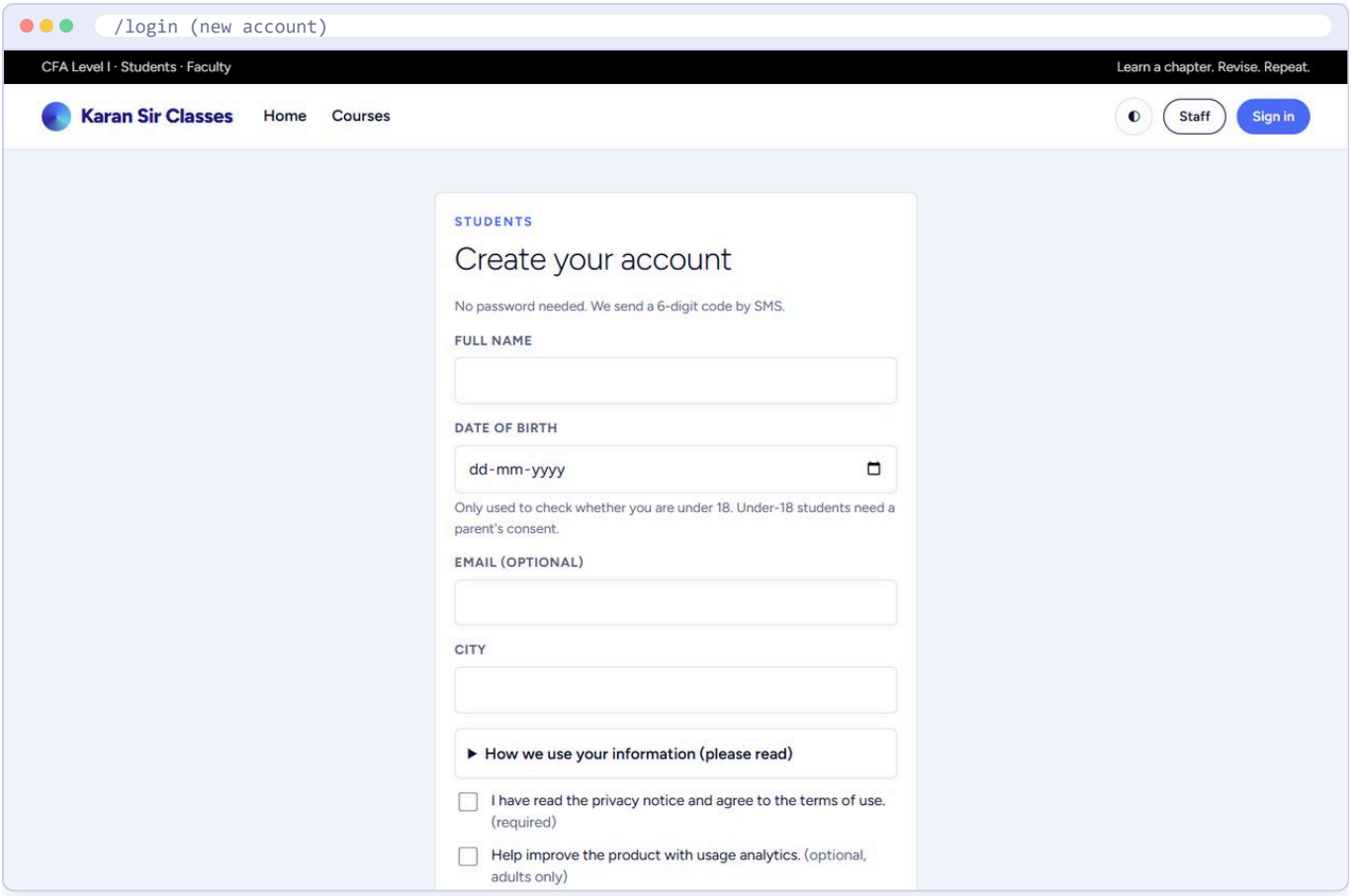


Figure 9. First-time candidates complete a short form. Date of birth is used only to decide whether guardian consent is needed.

Rule	Value	Source
Code validity	5 minutes, single use	`otp_ttl_seconds=300`
Code requests	5 per hour per mobile or email	`otp_max_requests_per_hour`
Wrong guesses	5 per code, then a new code is required	`otp_max_attempts`
Devices	2 active sessions; the oldest ends when a third signs in	`student_max_sessions` (BR-ACC-05)
Mobile format	Indian numbers only: +91 followed by 10 digits starting 6–9	`MOBILE_RE`

1.5 Workflow C2 · Under-18 candidates and guardian consent

Under India's DPDP rules, a child's data needs a parent's verifiable consent. A candidate whose date of birth gives an age below 18 gets status `pending_guardian`. They can sign in and see `/pending-consent` but cannot register, pay or study until the guardian approves.

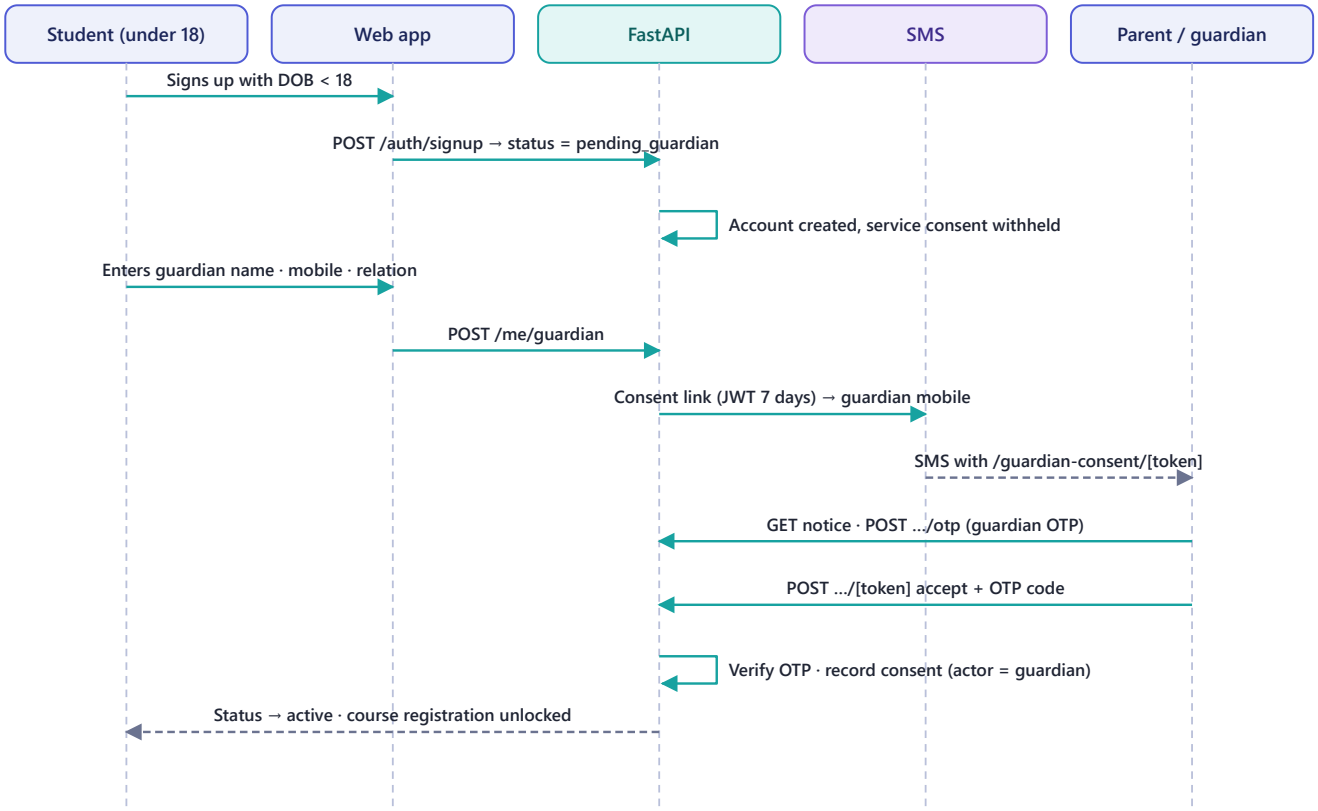


Figure 10. Guardian consent: the guardian proves identity with their own OTP; consent is stored with actor = guardian.

1.6 Workflow C3 · Choosing, registering and paying

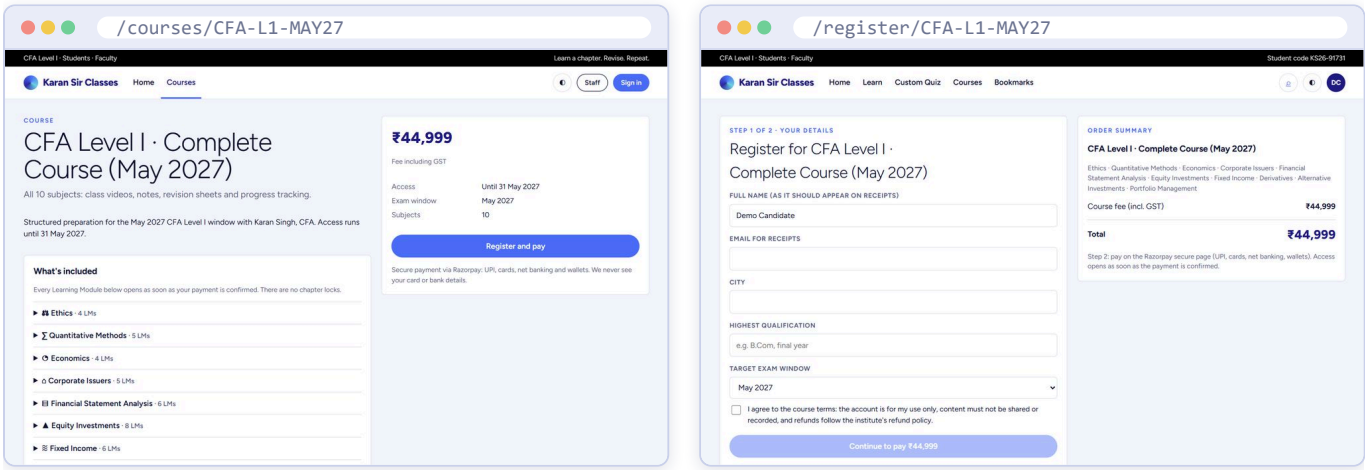


Figure 11. Course detail with fee and inclusions, then the registration form (name, email, city, qualification, target exam).

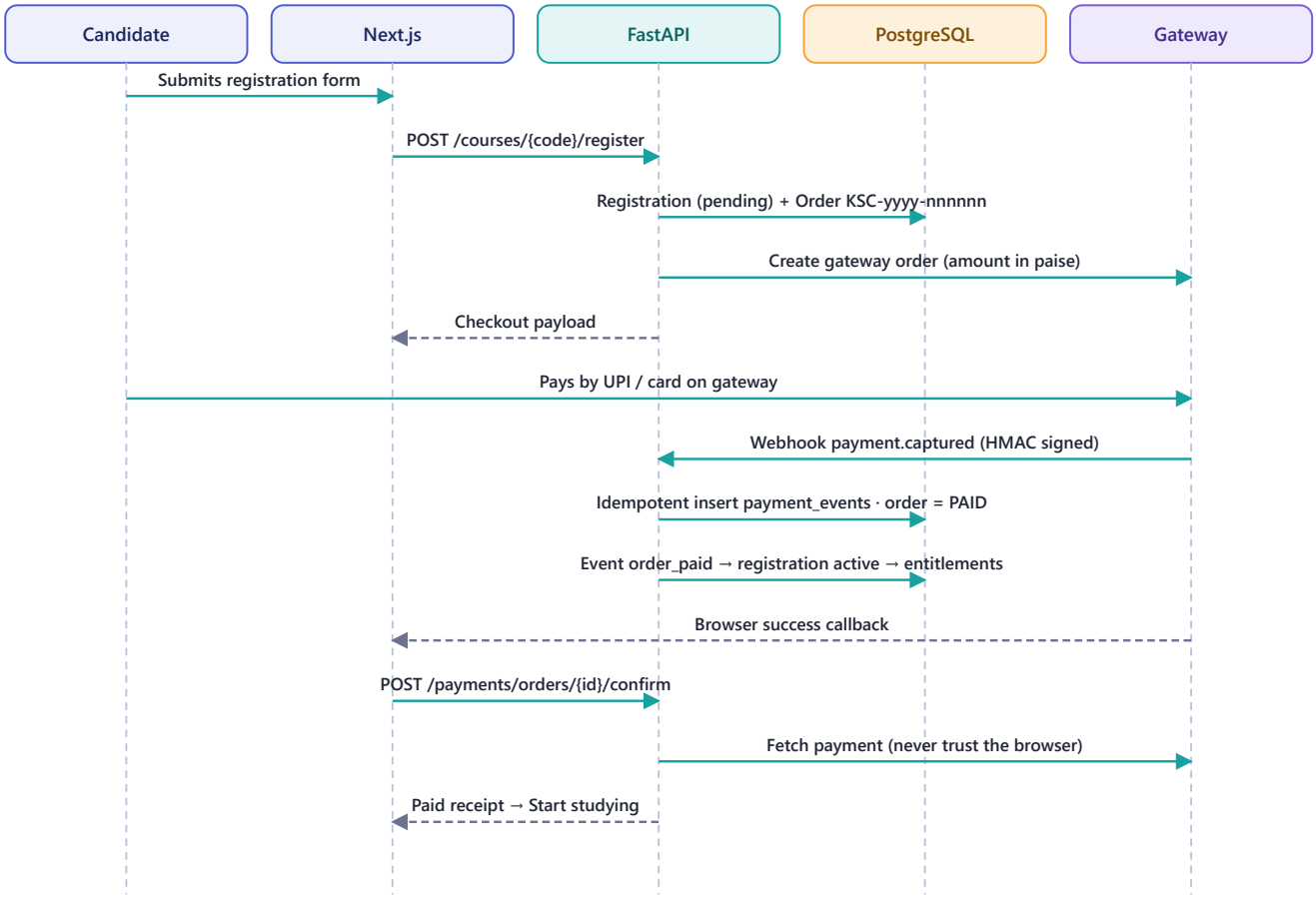


Figure 12. Registration and payment. Access opens when the signed gateway webhook arrives; the browser callback is only a convenience and is re-verified with the gateway.

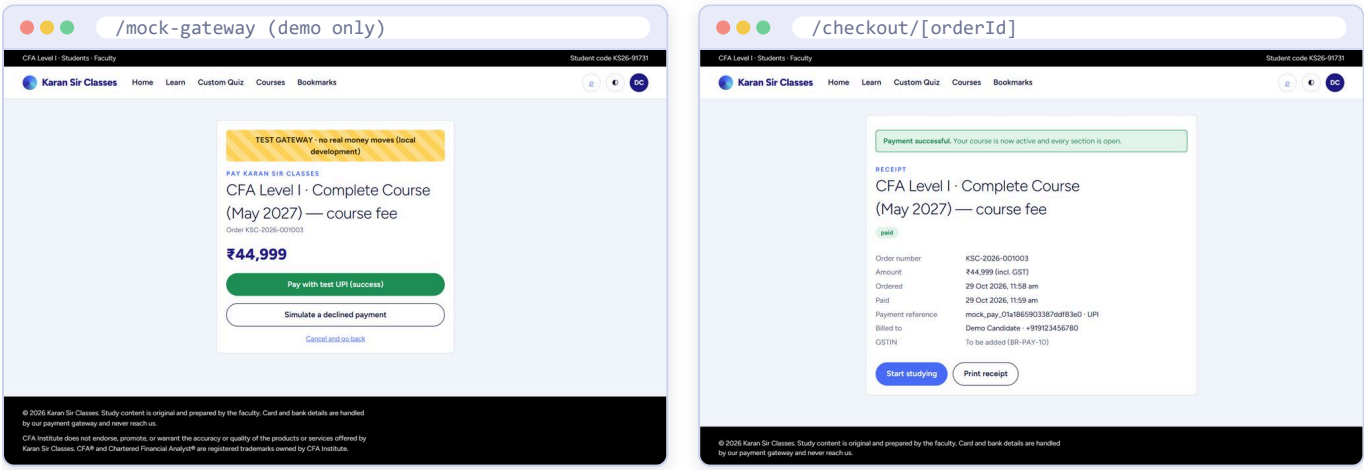


Figure 13. Demo gateway and the receipt page. In production the candidate pays on Razorpay's hosted checkout; the receipt page and numbering (KSC-year-number) are identical.

Why this is safe

Orders are idempotent (the same payment applied twice changes nothing), amounts are compared to the order (mismatch is flagged, never granted), and an unpaid order is reused for 30 minutes so a refresh cannot create duplicate charges.

1.7 Workflow C4 · Learning

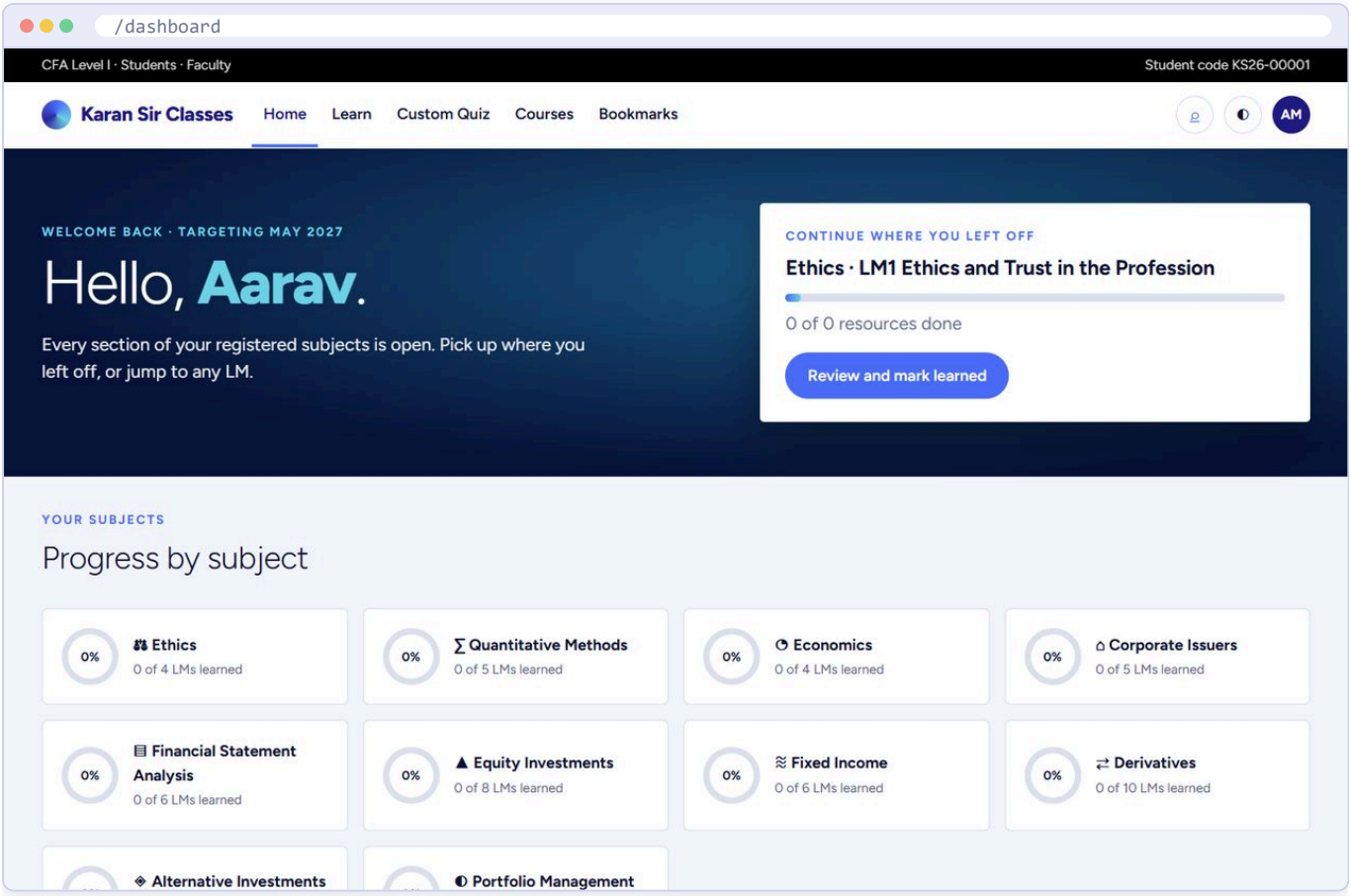


Figure 14. Dashboard: continue where you left off, plus a progress ring for each of the 10 subjects.

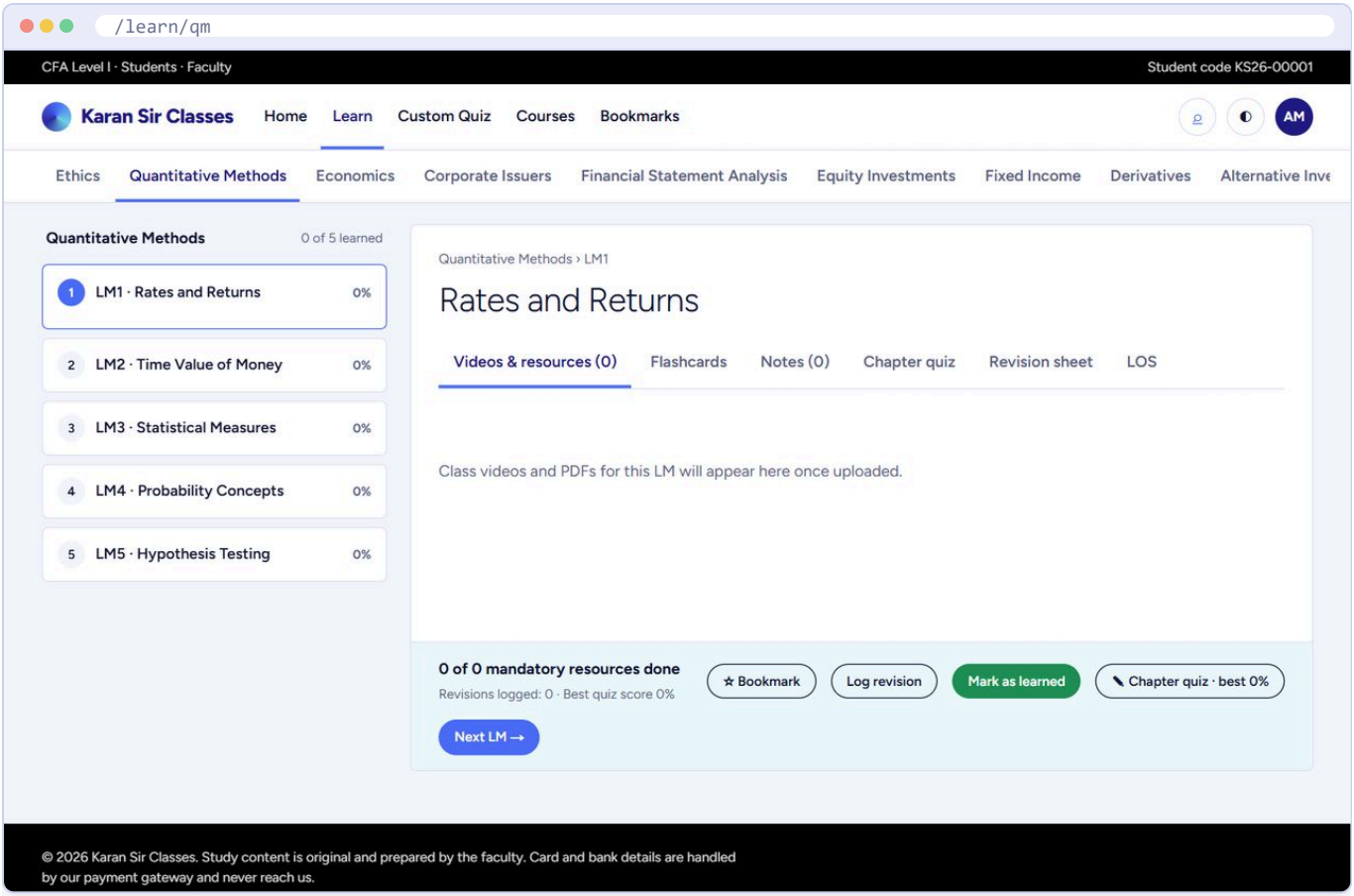


Figure 15. LM view: lessons on the left, tabs for resources, flashcards, notes, chapter quiz, revision sheet and LOS; action bar for bookmark, revision log, mark as learned.

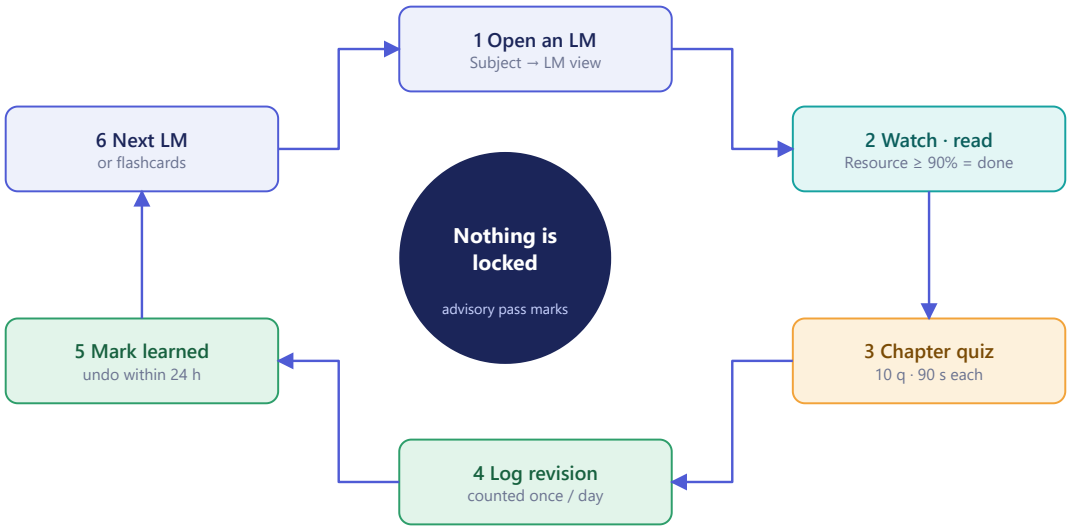
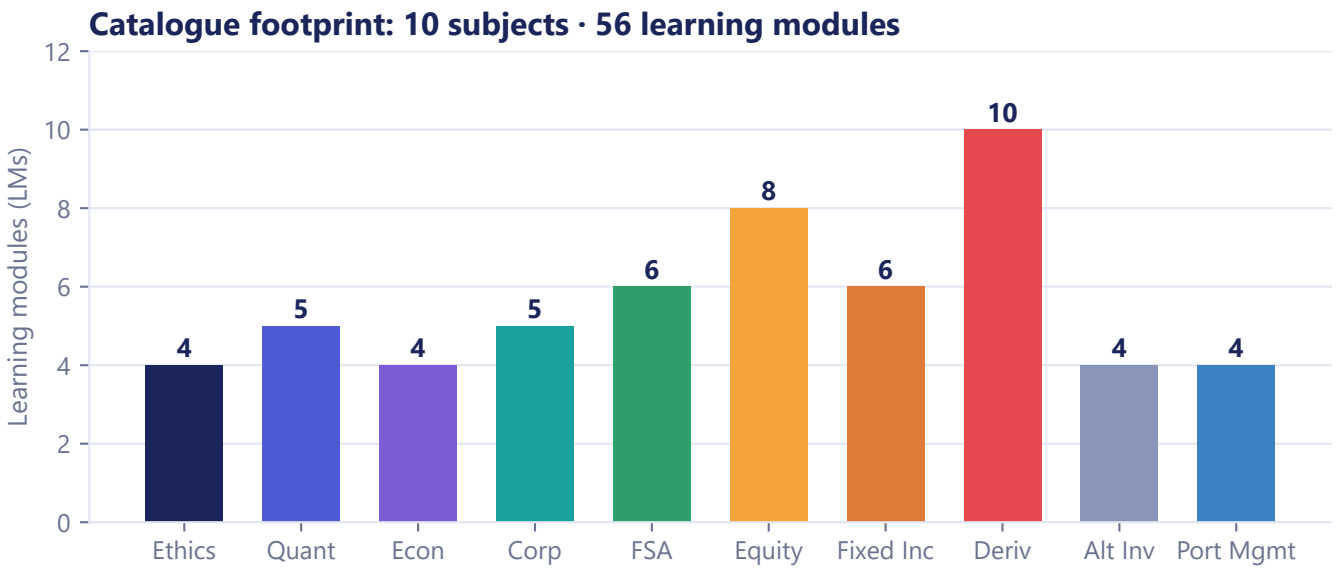


Figure 16. The learning loop. No step blocks another: pass marks are advice, not gates.

Behaviour	Rule	Where enforced
Resource completion	A video or PDF counts as done at ≥ 90% watched (merged watched intervals, so skipping back does not double count)	`progress/service.py` COMPLETION_THRESHOLD
Mark as learned	Allowed any time; can be undone for 24 hours	`lm_progress.learned_undo_until`
Revision log	Counted once per LM per day, so spamming the button does not inflate numbers	unique index `uq_revision_counted_per_day`

Behaviour	Rule	Where enforced
Access to content	An LM opens only if the candidate holds an active entitlement for its subject	`enrolment.assert_entitled`
Video / PDF delivery	Signed link that expires in 2 hours; a watermark with student code and last 4 digits of mobile is shown	`/resources/{id}/playback`



1.8 Workflow C5 · Tests and quizzes

Test	Questions	Timing	Advisory pass mark	Started from
Chapter quiz	10 (min 3 available)	90 s per question	80%	LM view → Chapter quiz
Subject (master) test	50	90 s per question	75%	Custom Quiz → Subject tests
Custom quiz	Your choice	Optional timer	none	Custom Quiz: subjects, chapters, difficulty (E/M/H), conceptual / numerical, practice or assessment

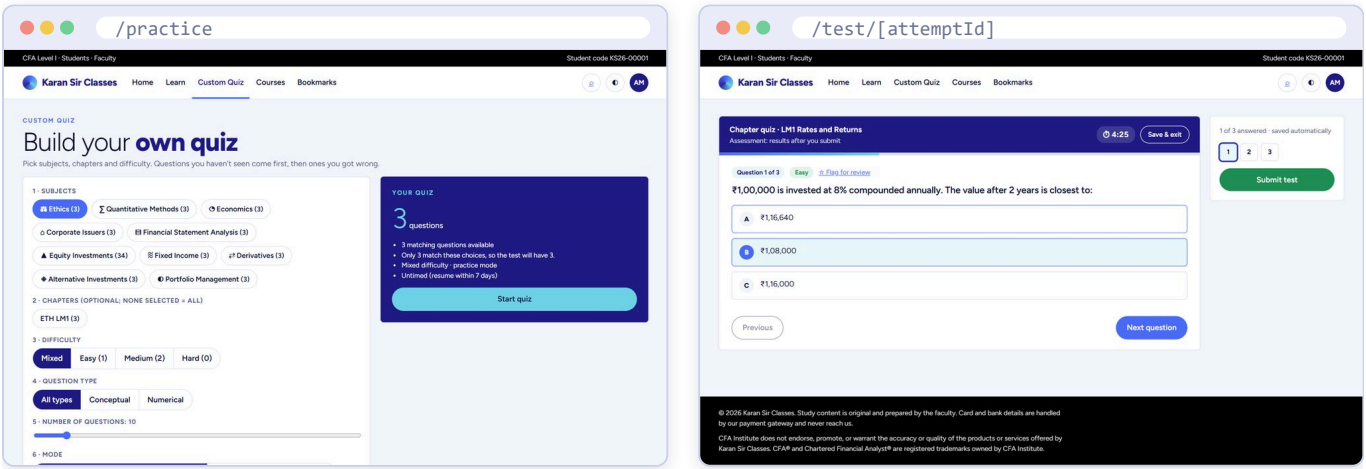


Figure 17. Custom quiz builder with live pool counts, and the attempt screen: timer, question palette, flag for review, autosave.

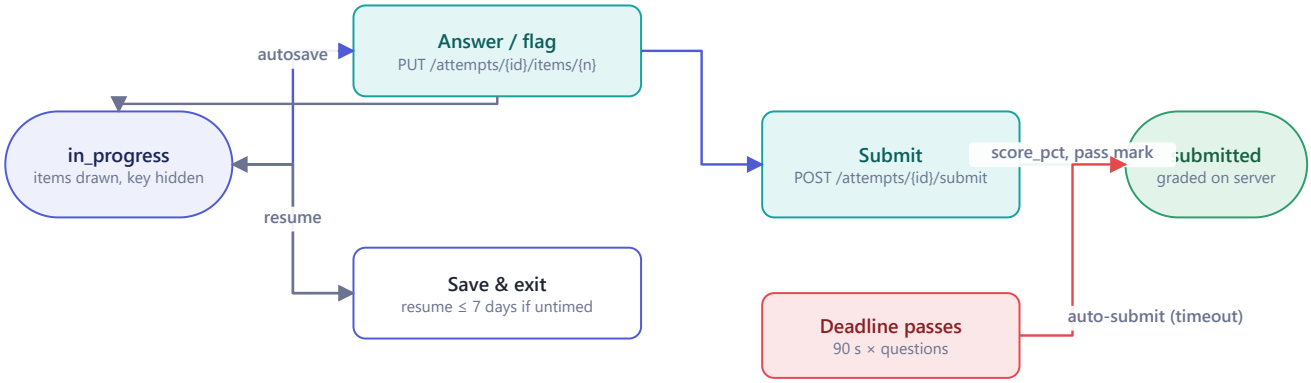


Figure 18. Attempt life cycle. Answers autosave; the server grades; a missed deadline auto-submits.

> **Answer keys stay on the server**
> In assessment mode the correct answer and explanation are not sent to the browser until that question is submitted. The attempt stores a snapshot of each question, so later edits by staff never change a past result.

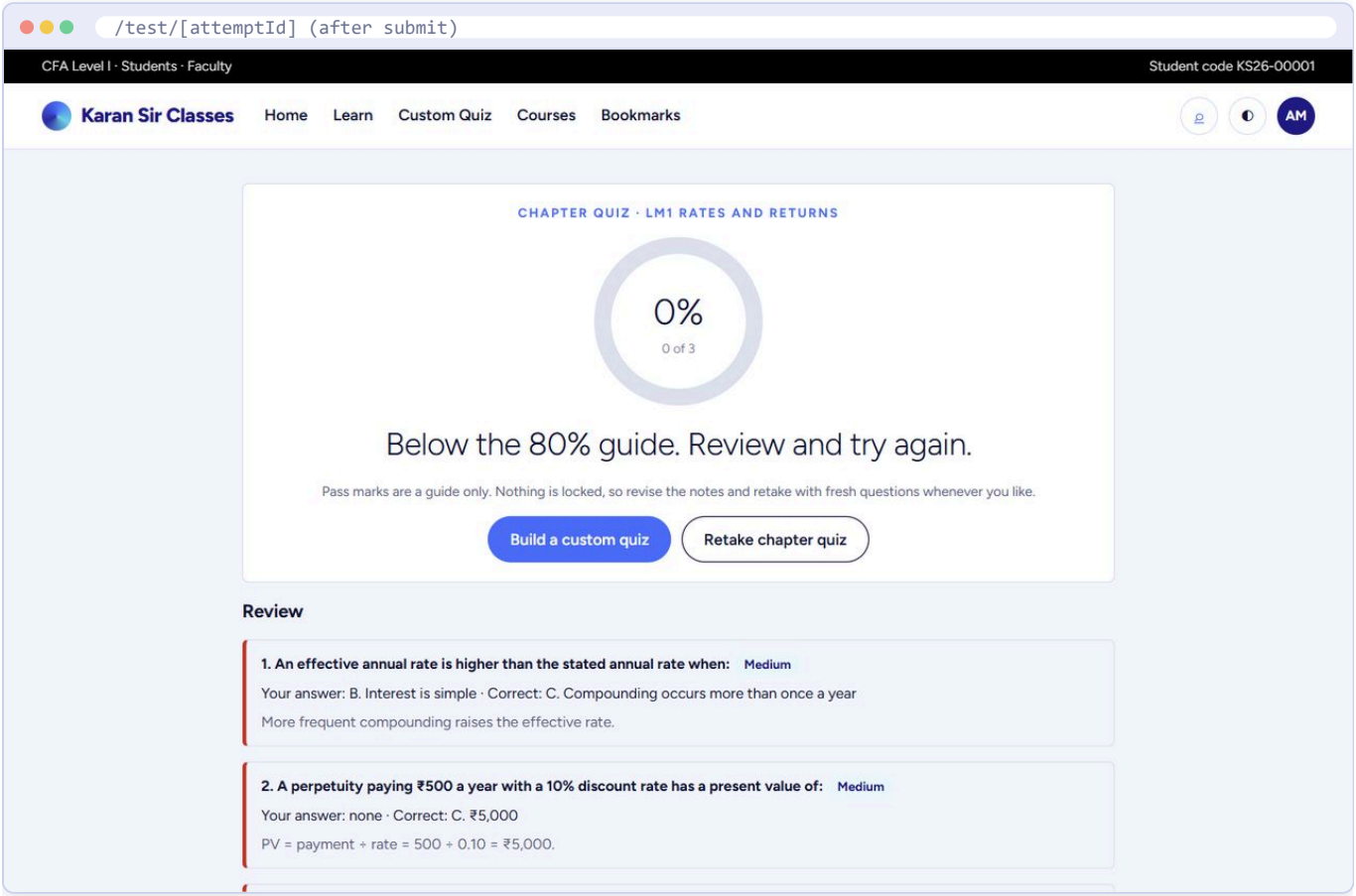


Figure 19. Result screen: score, pass-mark guidance, review of each question with explanation, and a retake that draws unseen questions first.

1.9 Workflow C6 · Flashcards and spaced repetition

Cards are grouped into decks by subject. After flipping a card the candidate rates it **Known**, **Needs revision** or **Difficult**. A modified SM-2 scheduler sets the next due date.

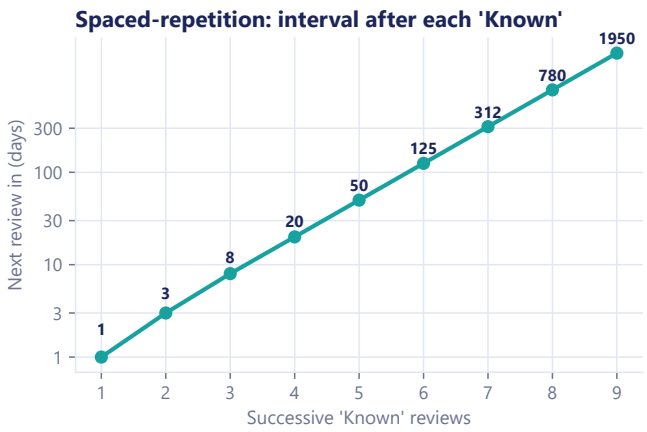


Figure 20. Interval grows after every *Known*: 1, 3, then roughly × ease (2.5).

Rating	Effect
Known	Interval 1 d → 3 d → previous × ease
Needs revision	Interval × 1.2, ease − 0.15
Difficult	Back to 1 day, ease − 0.2, counts a lapse
Floor	Ease never below 1.3

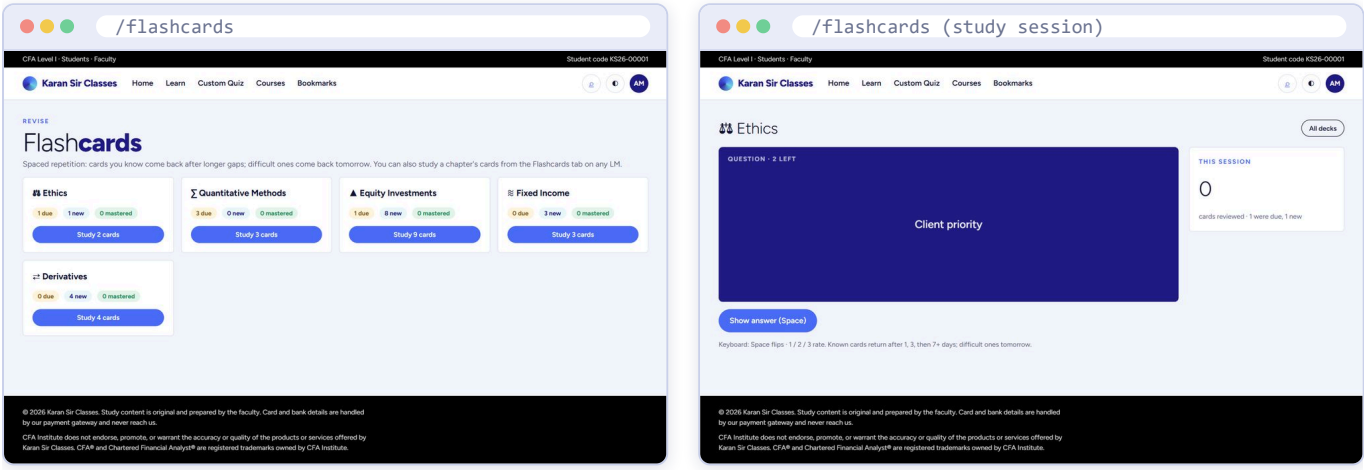


Figure 21. Deck list with due counts and the study card.

1.10 Workflow C7 · Search, bookmarks, orders and profile

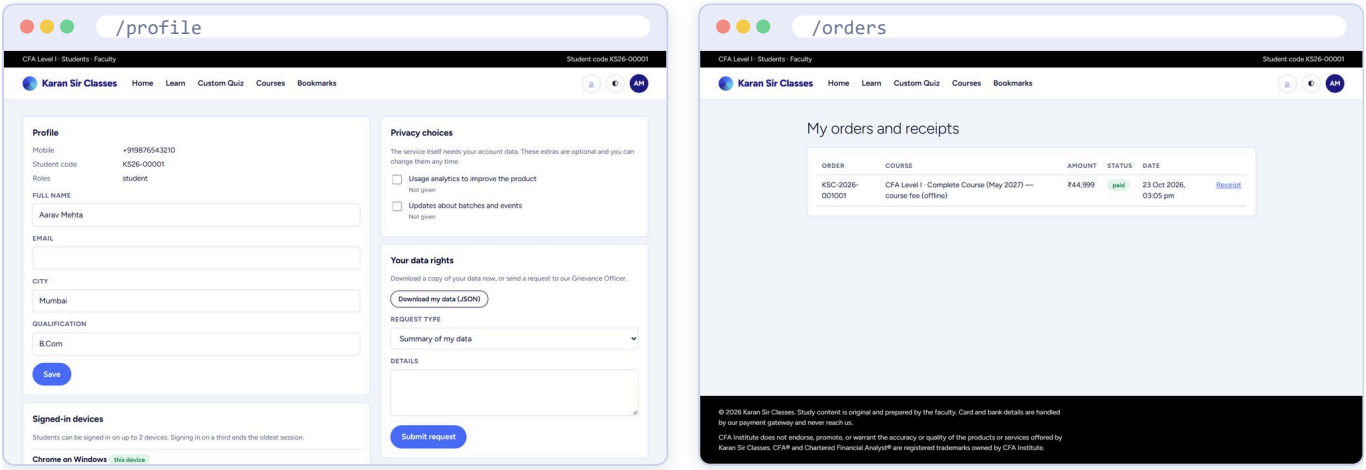


Figure 22. Profile page: details, signed-in devices (end any session), privacy choices, data-rights requests. Orders page: every receipt.

- **Search** (`/search?q=`) finds learning modules (by title or summary) and published notes (PostgreSQL full-text search). Results outside the candidate's subjects show the title but no text snippet.
- **Bookmarks** save a note, a video position or an LM with an optional note (500 characters).

- **Devices** lists active sessions with a device label such as “Chrome on Windows”; ending one revokes its refresh token immediately.

1.11 Staff and admin workflows

Workflow S0 · Signing in to the console

Staff sign in at `/staff/login` with email and password, then a six-digit authenticator code. First-time staff are walked through enrolling an authenticator app. Outside local development, MFA is mandatory and the server refuses to start without it.

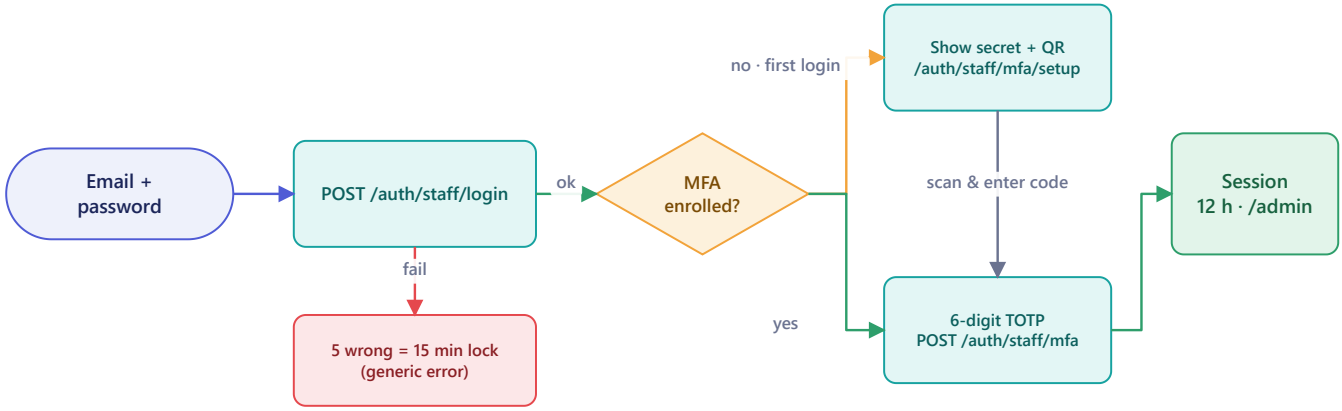


Figure 23. Staff sign-in with enforced second factor.

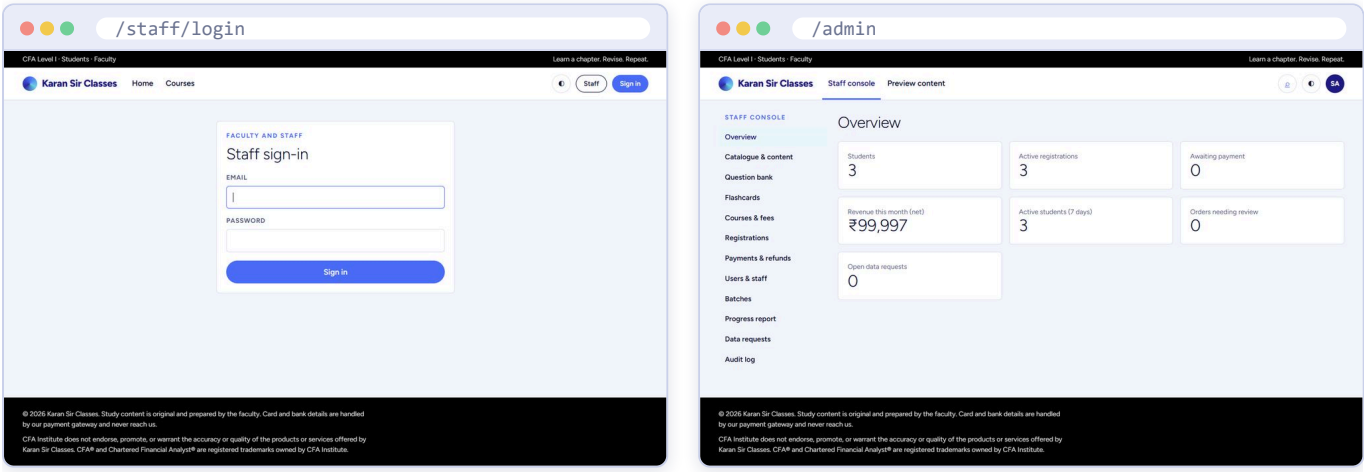


Figure 24. Staff sign-in and the overview dashboard (students, active registrations, revenue, open data requests).

Workflow S1 · Content and question lifecycle

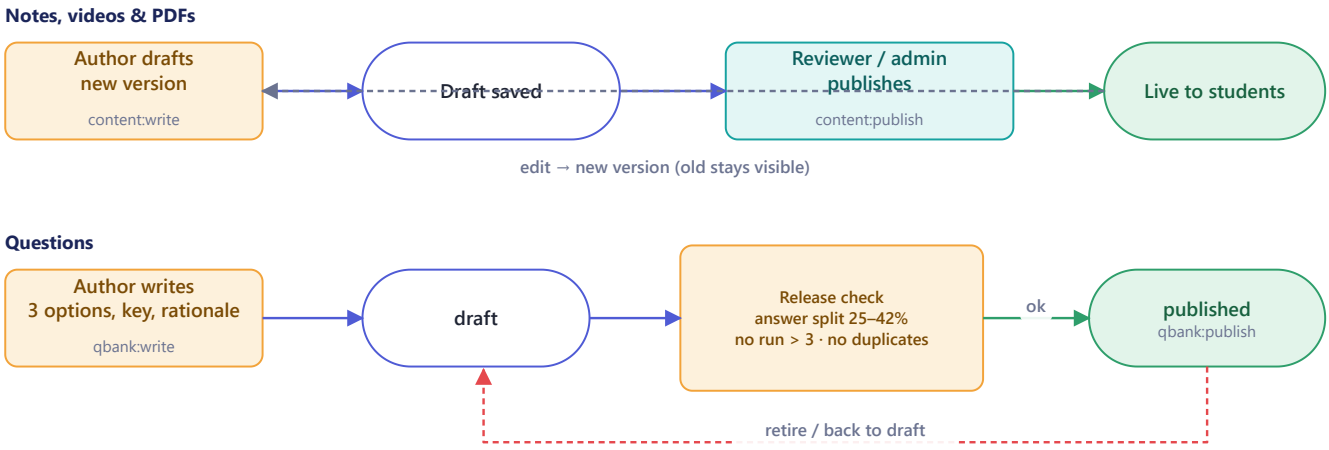
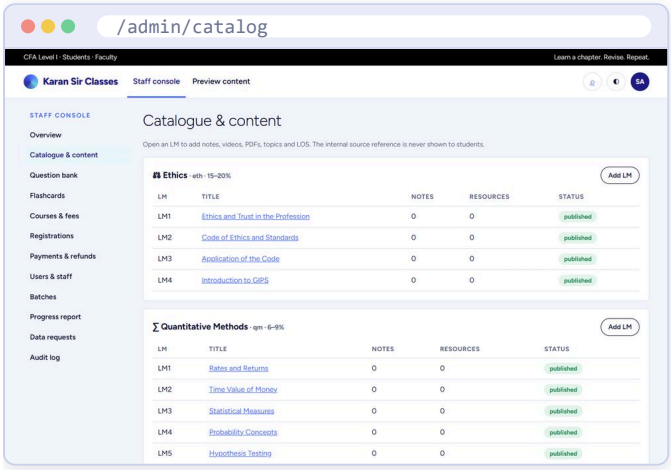


Figure 25. Separation of duties: authors write, reviewers and admins publish. Published content is versioned; a new version never replaces what students see until it is published.



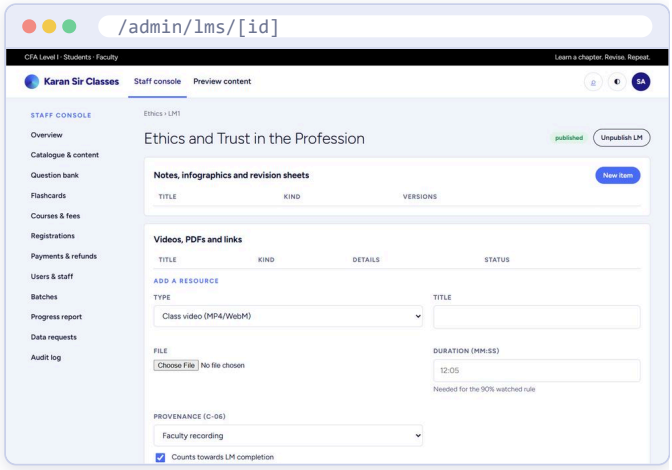
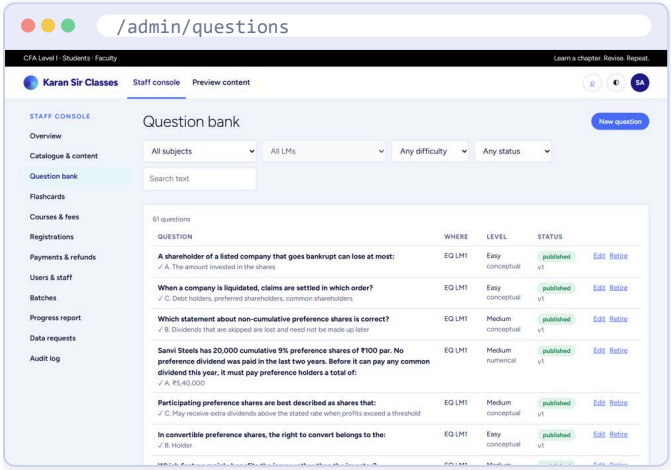


Figure 26. Catalogue list per subject and the LM editor (topics, learning outcome statements, notes, video and PDF upload).



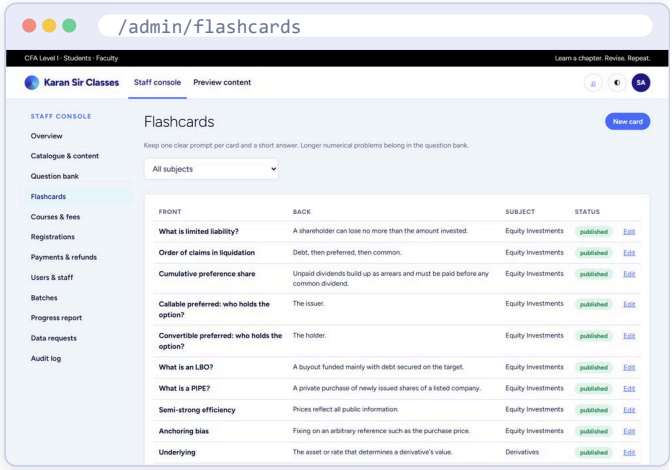


Figure 27. Question bank and flashcard management.

> **Upload safety**
> Videos and PDFs are accepted only for allowed MIME types; PDFs must start with the `%PDF` signature; size is capped at 500 MB; licensed third-party material needs a licence reference. An antivirus scan step is planned before publish (not yet implemented).

Workflow S2 · Courses, registrations, payments

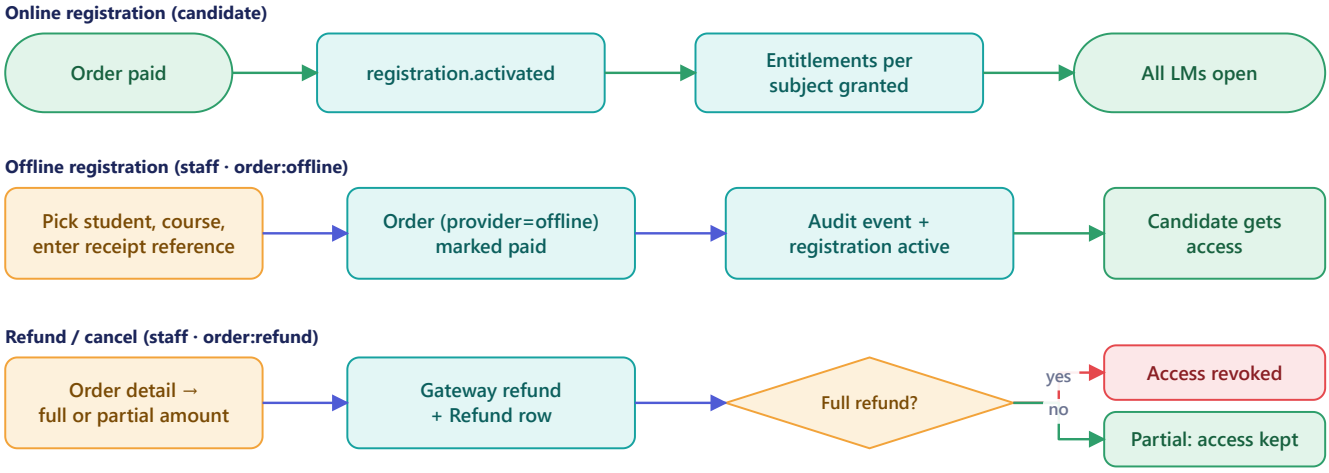


Figure 28. Three money flows an admin handles: online (automatic), offline (manual entry) and refund.

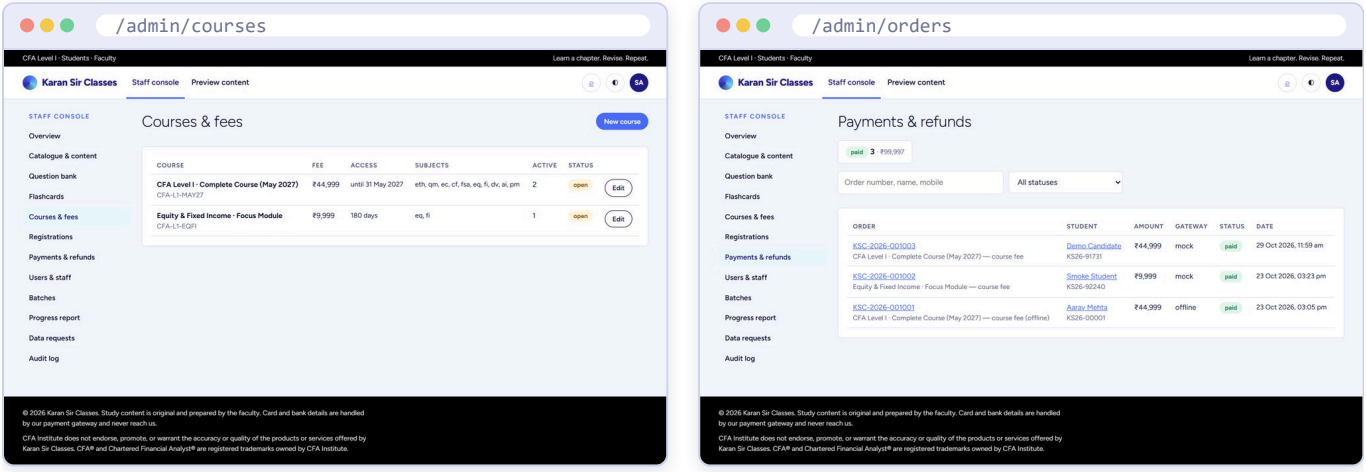


Figure 29. Courses and fees, and the orders list with status filters.

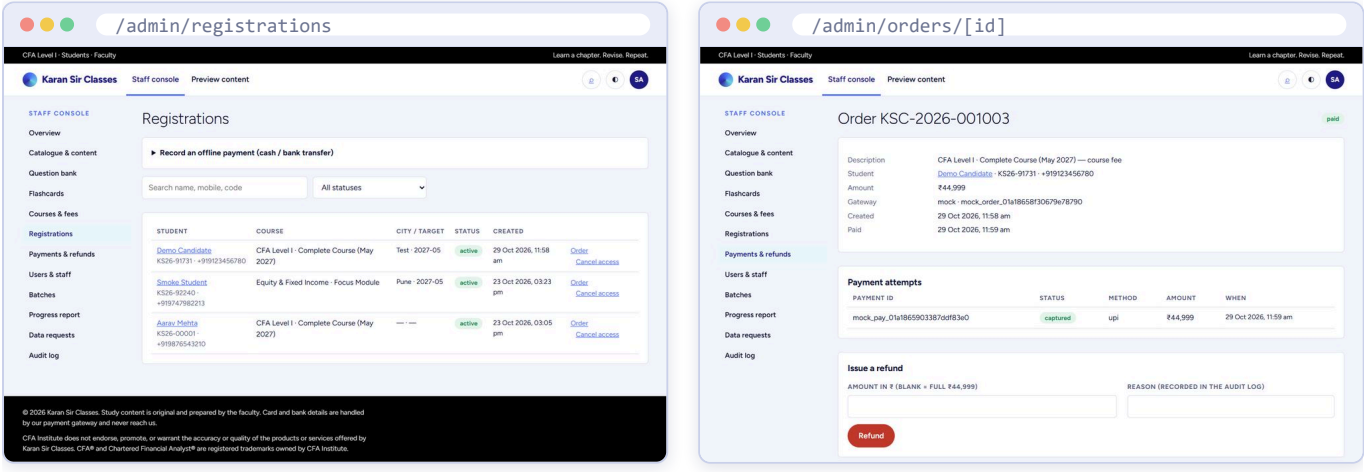
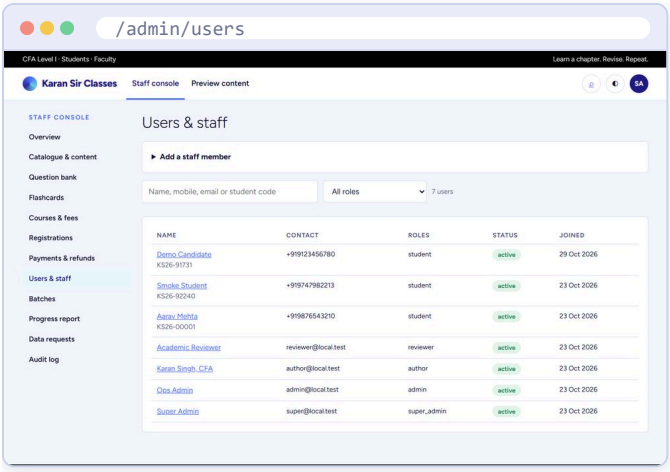


Figure 30. Registrations (including offline entry and cancel) and the order detail with refund action.

Workflow S3 · People, batches and access



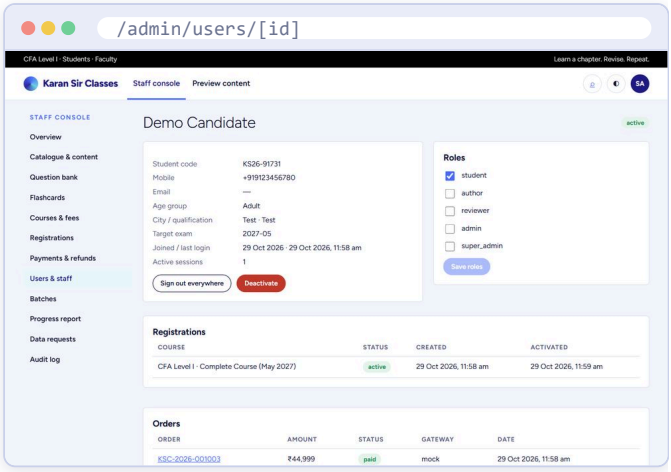
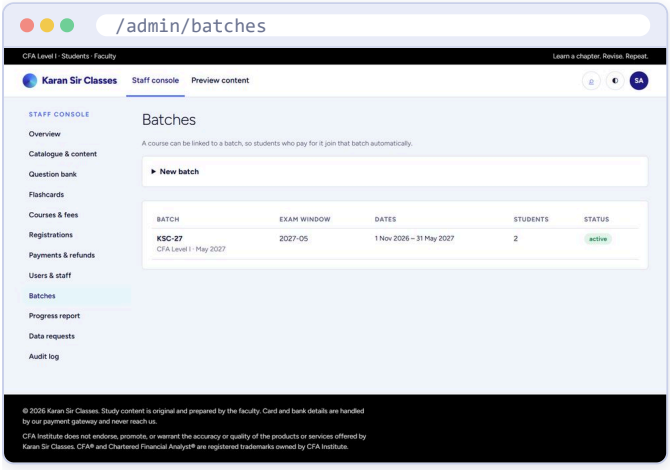


Figure 31. User directory and user detail: roles, entitlements (grant or revoke), sessions (force sign-out), deactivate.



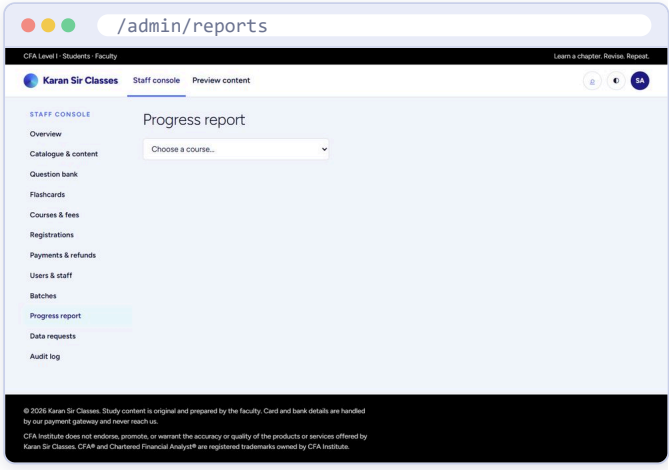


Figure 32. Batches (cohorts with start/end dates) and the progress report per course.

Workflow S4 · Privacy requests and audit

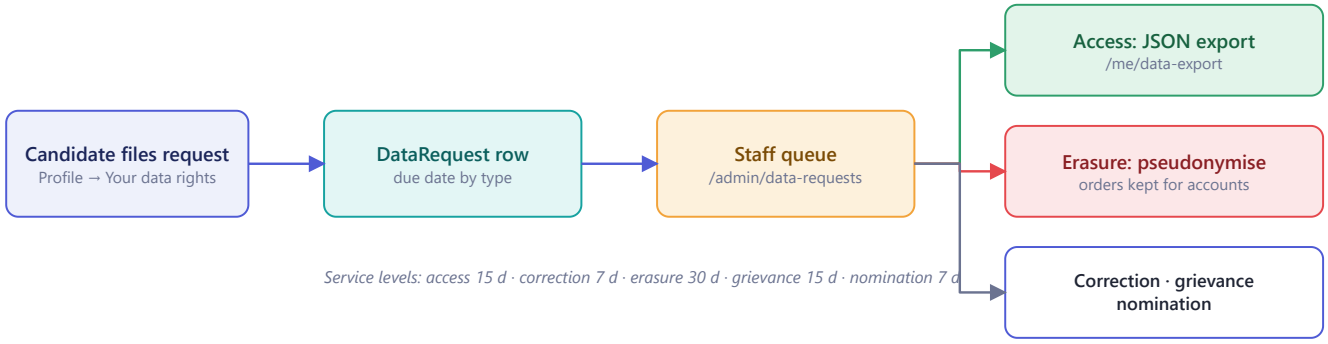


Figure 33. Data-subject requests under DPDP: five request types, each with a due date.

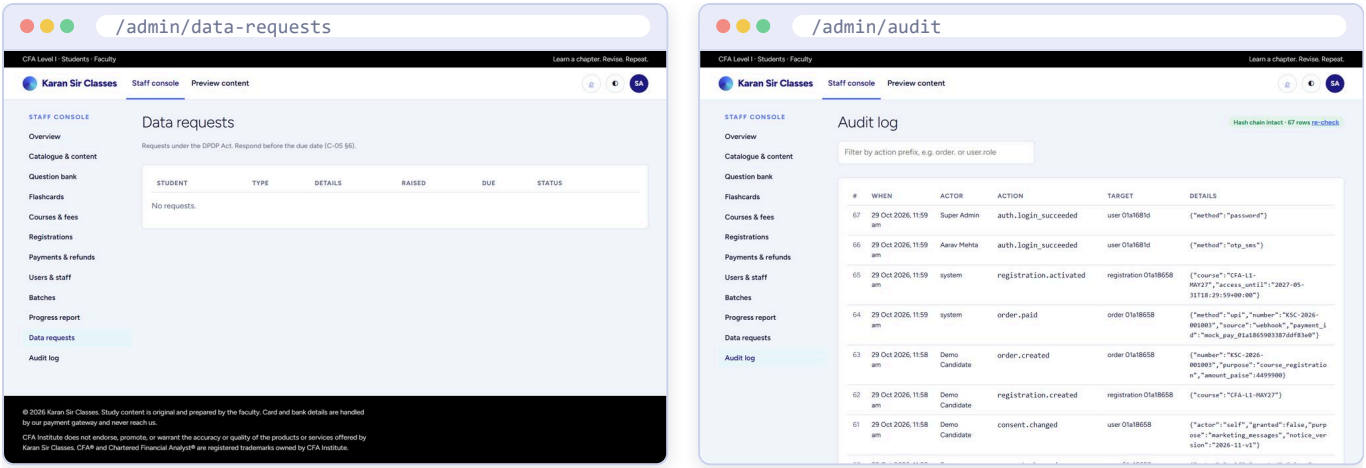


Figure 34. Data-request queue and the audit log. “Verify chain” recomputes the hash chain to prove no row was altered (see Part 5).

1.12 Who can open which screen

The staff menu is filtered by permission. This table is the contract between roles and screens; the same permissions guard the underlying API calls.

Screen	Permission	Author	Reviewer	Admin	Super
Overview · Progress report	report:read	–	✓	✓	✓
Catalogue & content · Question bank · Flashcards	catalog:read_admin	✓	✓	✓	✓
Courses & fees	course:manage	–	–	✓	✓
Registrations · Payments & refunds	order:read	–	–	✓	✓
Users & staff	user:read	–	–	✓	✓
Batches	batch:manage	–	–	✓	✓
Data requests	data_request:process	–	–	✓	✓
Audit log	audit:read	–	–	✓	✓

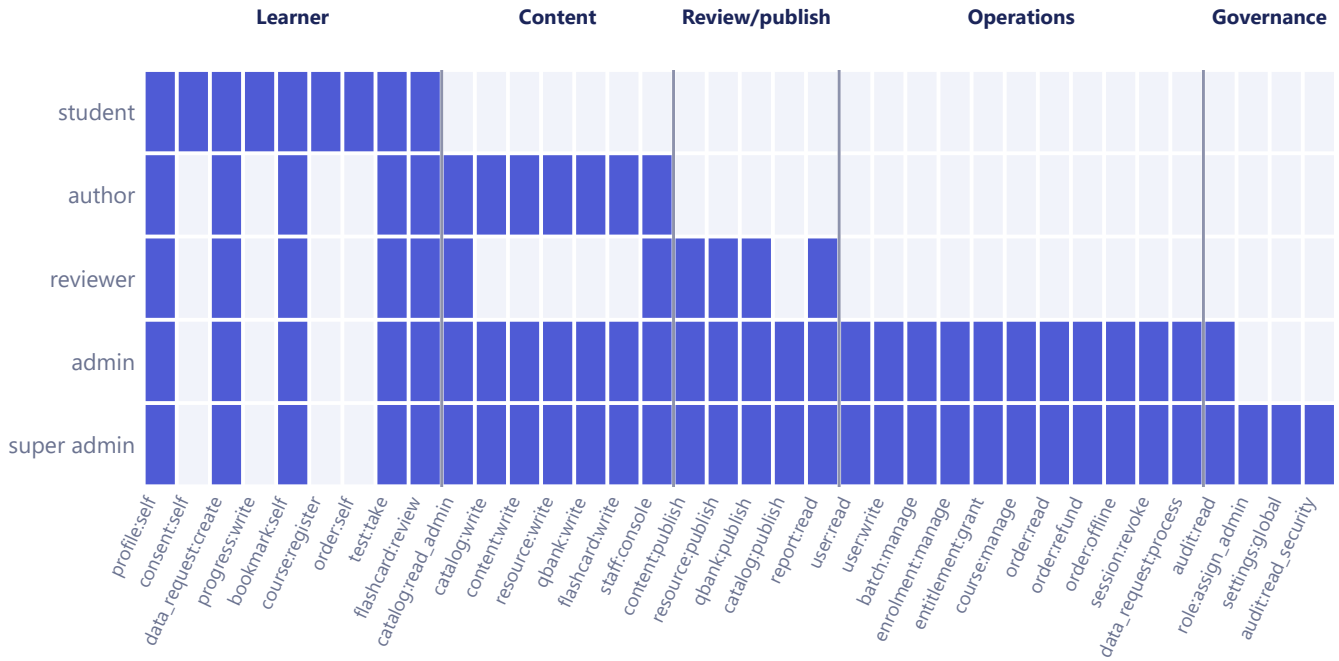


Figure 35. Full permission matrix: 32 permissions grouped by purpose, one row per role.

PART 2

Backend mapping

Tech stack, technical architecture, deployment and the shape of the code.

2.1 Technology stack

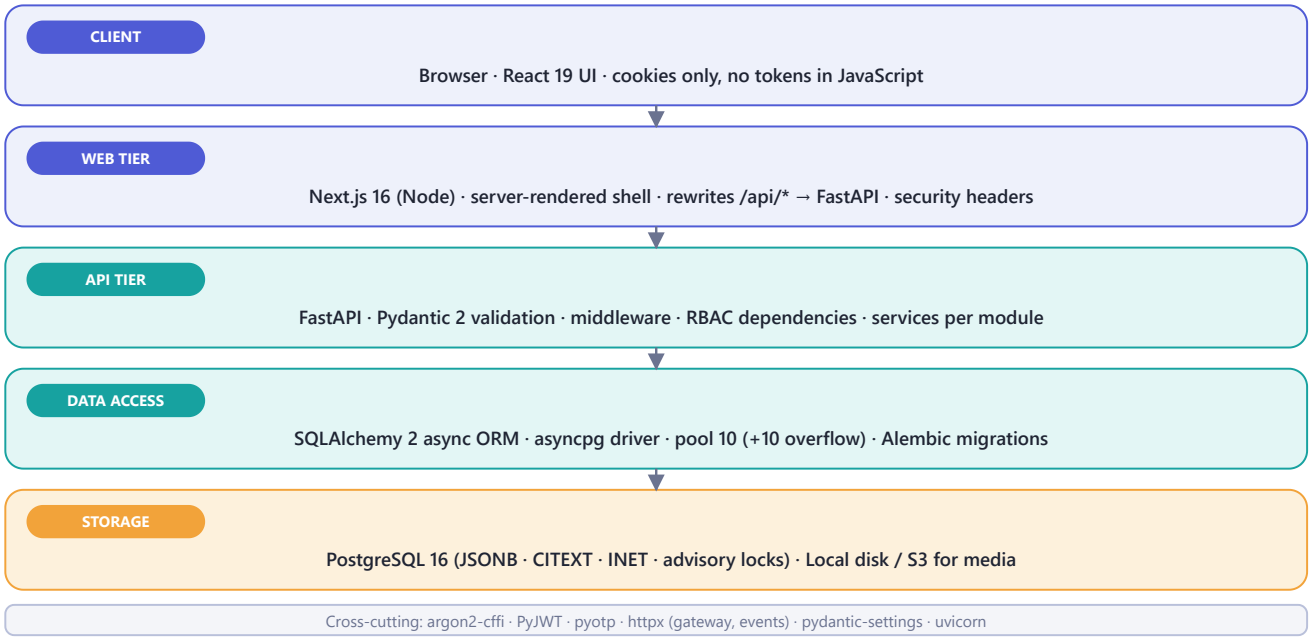


Figure 36. Five-layer stack from browser to storage.

Area	Library / service	Version constraint	Used for
Web framework	Next.js · React · TypeScript	^16.4 · ^19.3 · ~5.9	35 pages, same-origin proxy, security headers
API framework	FastAPI · Pydantic · Uvicorn	≥0.115 · ≥2.9 · ≥0.54	Routing, validation, ASGI server
ORM / DB	SQLAlchemy (async) · asyncpg · Alembic	≥2.0.36 · ≥0.32 · ≥1.20	Models, queries, migrations (2 revisions so far)
Database	PostgreSQL 16	extensions: citext, pg_trgm	37 tables, JSONB, INET, advisory locks
Security	argon2-cffi · PyJWT · pyotp	≥25.1 · ≥2.9 · ≥2.10	Password hashing, tokens, TOTP
HTTP client	httpx	≥0.28	Gateway and event-sink calls
Config	pydantic-settings	≥2.16	12-factor settings with start-up guard rails
Quality	pytest · pytest-asyncio · ruff · tsc	≥9.1 · ≥1.4 · ≥0.17	32 tests, lint (E,F,I,B,UP), type-check

2.2 Technical architecture: a modular monolith

The API is one deployable service organised into modules. Each module owns its models, service functions and router. Modules talk through service calls and domain events, never by reaching into each other's tables, which keeps a later split into separate services possible without a rewrite (the payments module is already written to be replaceable by an external service).

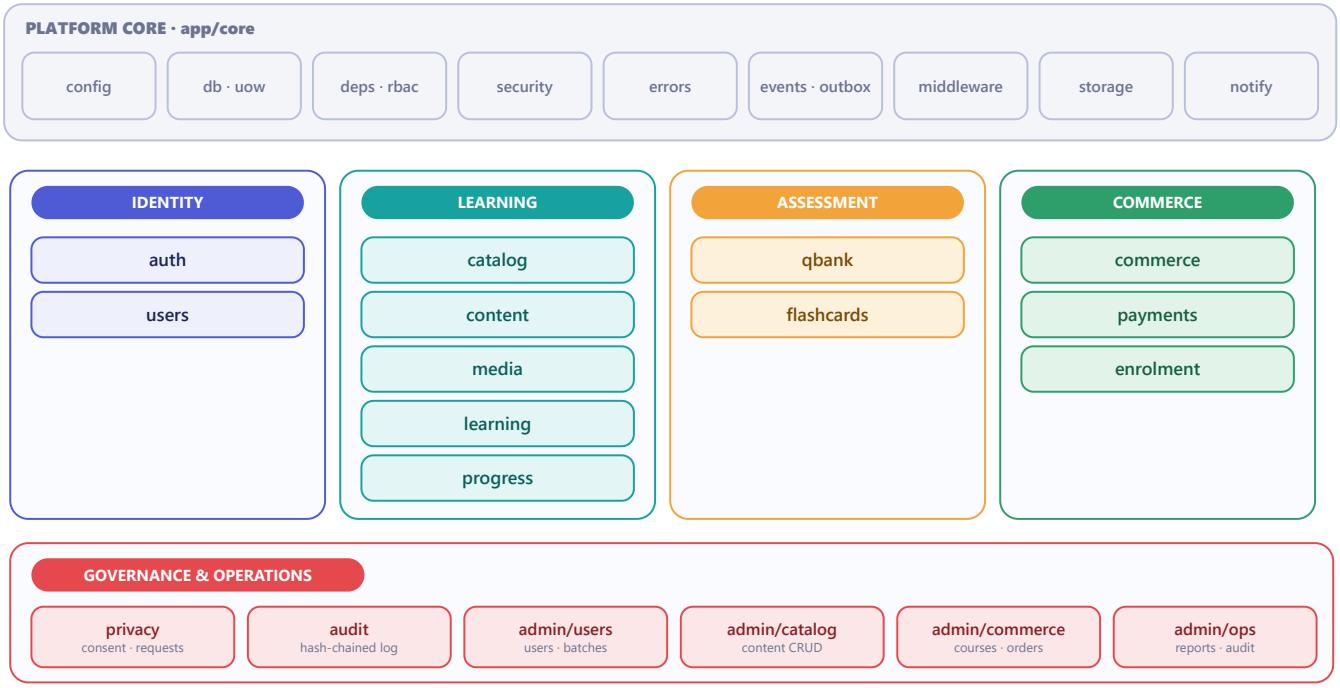


Figure 37. Module map. Every box is a Python package under `app/modules` (or `app/core`).

Module	Owns tables	Responsibility
auth	otp_challenges, sessions	OTP, staff login, TOTP, session rotation
users	users, user_roles, guardians	Identity records
catalog · content · media	subjects, learning_modules, topics, los · content_* · resources	What there is to learn, versioned, with files
learning · progress	lm_progress, resource_progress, revision_logs, eoc_entries, bookmarks	Reading content, tracking progress, search
qbank	questions, test_attempts, attempt_items	Question bank, tests, grading
flashcards	flashcards, card_reviews	Cards and spaced repetition
commerce · enrolment	courses, registrations, batches, enrolments, entitlements	Selling a course and granting access
payments	orders, payments, payment_events, refunds	Gateway adapters (Razorpay, mock), webhooks, refunds
privacy · audit	consents, data_requests, audit_log, outbox, settings	DPDP compliance, tamper-evident log, events

2.3 Request lifecycle

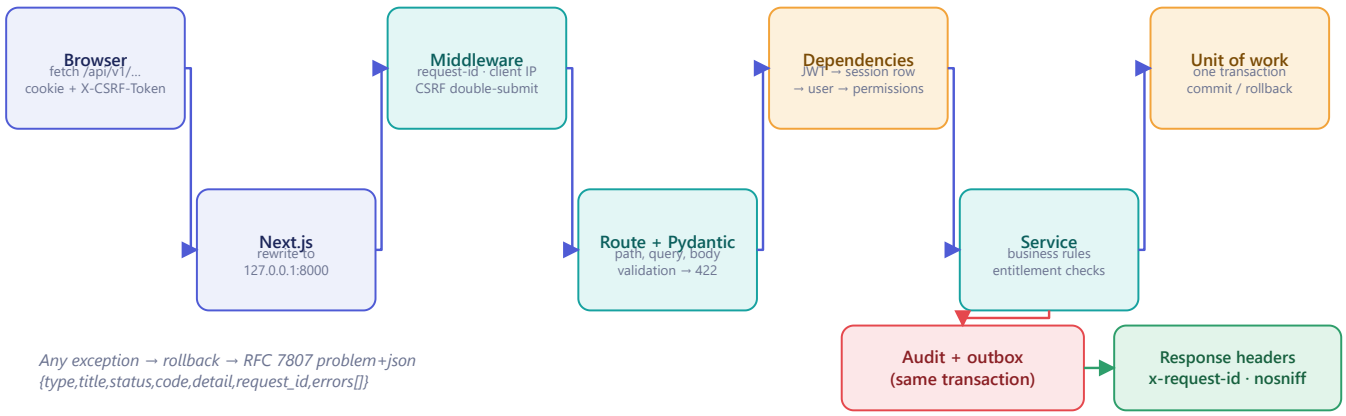


Figure 38. What happens on every API call. The unit of work commits once at the end, so a failed audit write rolls back the business change too.

Concern	How it is done	Code
Identifiers	UUIDv7: time-ordered (good for index locality), random part makes them unguessable	`core/ids.py`
Time	Timestamps are stored timezone-aware in UTC; business dates use IST	`core/timeutil.py`
Errors	RFC 7807 `application/problem+json` with a stable `code` field the UI switches on	`core/errors.py`
Validation	Pydantic models reject unknown fields and return per-field messages (HTTP 422)	`core/schemas.py`
Naming	Deterministic constraint and index names so migrations are reviewable	`core/db.py` NAMING
Settings	Environment variables; start-up asserts forbid unsafe combos outside dev	`core/config.py`
Notifications	SMS and email behind provider interfaces; console provider in dev, MSG91 / SES adapters planned	`core/notify.py`
Files	Storage interface with path-traversal protection and size cap; local disk now, S3 planned	`core/storage.py`

2.4 Domain events and the outbox

Payments finishing must open access, and a full refund must close it. These cross-module reactions use a small event bus. `publish()` writes the event to an `outbox` table *and* calls in-process subscribers inside the caller's transaction, so local consistency is immediate and atomic. A background loop then forwards pending events to an optional HTTP sink, signed with HMAC, so another team's service (CRM, analytics) can listen without touching this code.

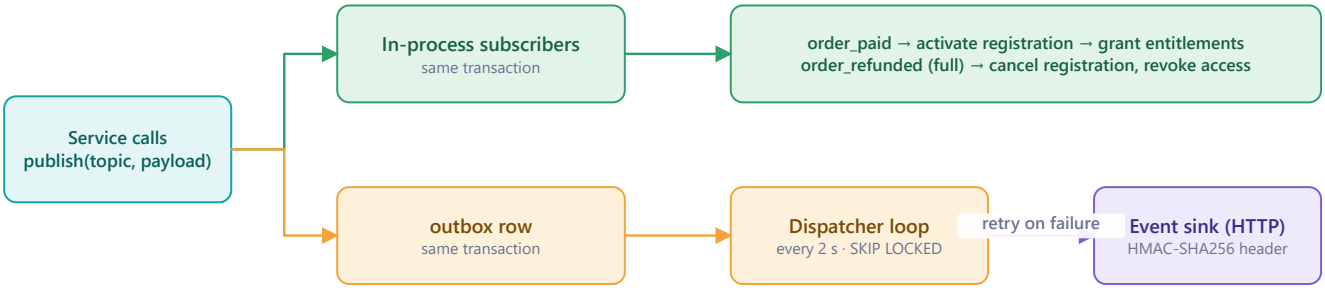


Figure 39. Transactional outbox. The dispatcher uses `FOR UPDATE SKIP LOCKED`, so several API replicas can run the loop safely.

2.5 Deployment today: one container on Render

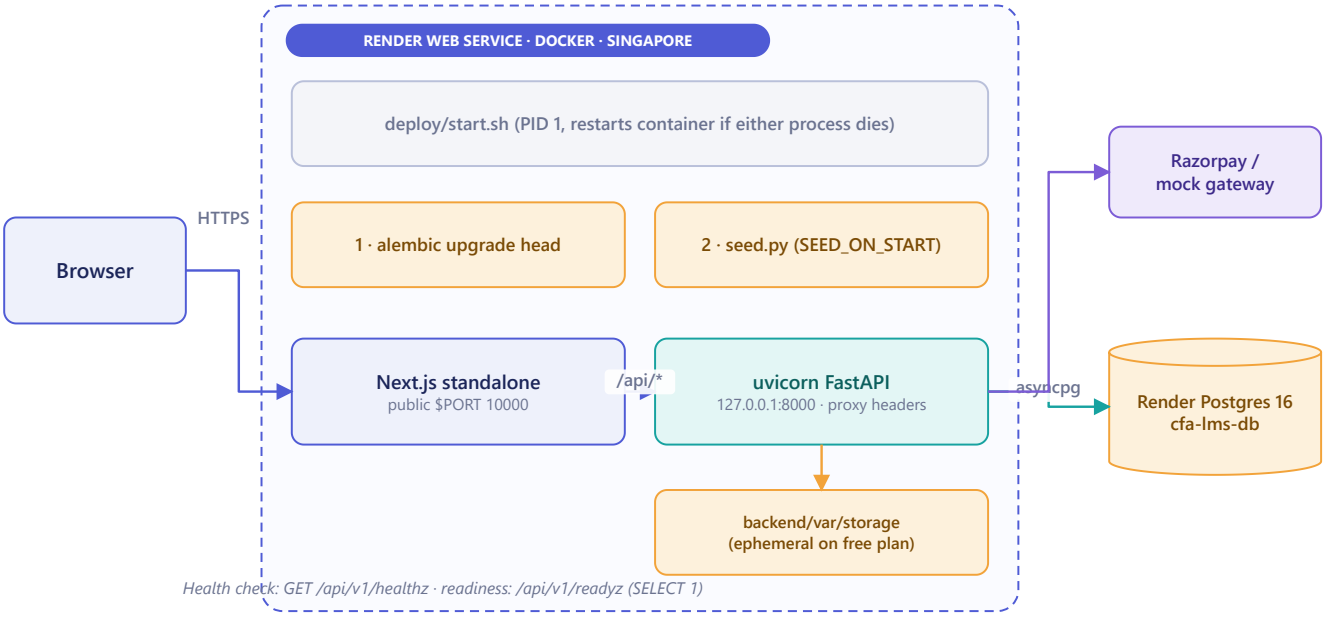


Figure 40. Current staging deployment from `render.yaml` and `Dockerfile`.

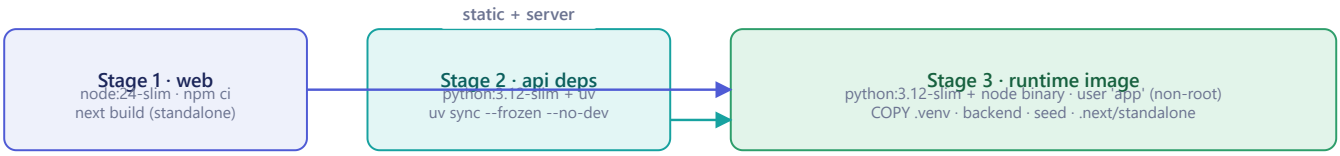


Figure 41. Three-stage image keeps build tools out of the runtime image; the process runs as a non-root user.

Variable	Staging value	Purpose
APP_ENV	staging	Activates production-style guard rails
DEMO_MODE	true (staging only)	On-screen OTP and mock gateway until SMS and Razorpay are connected; production refuses to start with it
COOKIE_SECURE · STAFF_MFA_REQUIRED	true · true	Mandatory outside local/test; server asserts at start-up

Variable	Staging value	Purpose
PAYMENT_PROVIDER	mock → razorpay	Selects gateway adapter
SECRET_KEY	generated by Render	Signs every JWT and keyed hash (min 32 chars)
DATABASE_URL	from Render Postgres	Auto-rewritten to the asyncpg driver
SEED_ON_START · SEED_STAFF_PASSWORD	true · prompted	Idempotent demo catalogue and four staff accounts

Free-plan limits to know
Render's free web service sleeps after 15 minutes idle (first request takes about a minute), has no persistent disk (uploaded PDFs vanish on redeploy) and its free Postgres expires after 30 days. Move to a paid plan, or to S3 for uploads, before real students sign up.

2.6 Target architecture: AWS Mumbai

The infrastructure document (A-05) defines where the platform is heading once traffic and compliance needs outgrow a single container. The application code is already stateless apart from the media directory, so the move is mostly infrastructure.

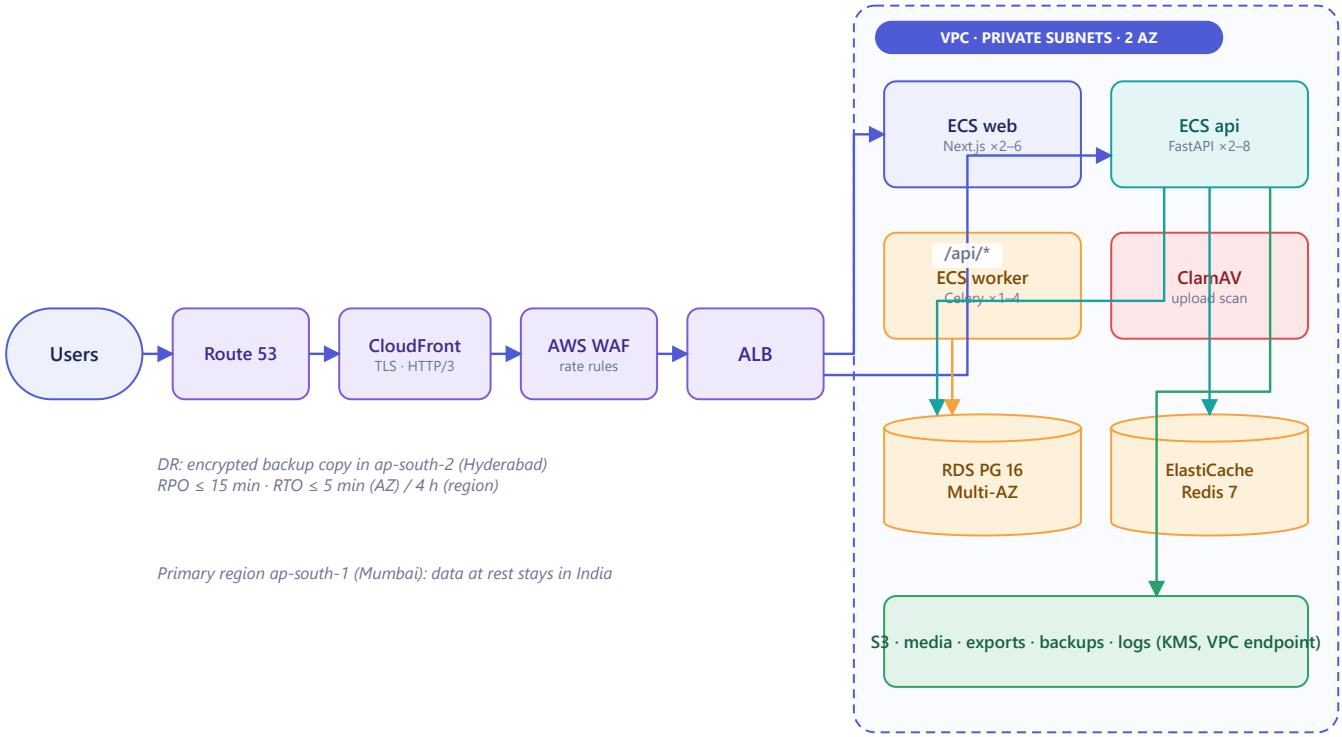
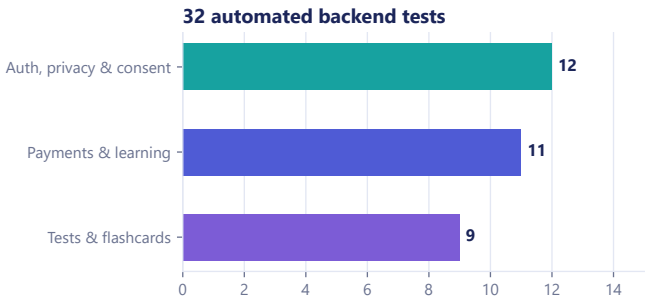


Figure 42. Target architecture from A-05 (design, not yet built). Elements not yet present in code: Redis, Celery worker, S3 adapter, ClamAV scan.

2.7 Quality and delivery



- Backend tests run against a separate `cfa_lms_test` database with real PostgreSQL features (advisory locks, JSONB).
- Covered: OTP and session rules, staff login and MFA, privacy and consent, payments, webhooks and refunds, learning progress, tests and flashcards.
- Frontend: `tsc --noEmit` and `next build` must pass.
- CI/CD design (D-02) and branching (D-03) are documented; pipeline files are not yet in the repo.

PART 3

APIs, database & UI mapping

Every endpoint, every table, and how each screen connects to them.

3.1 API conventions

Topic	Convention
Base path	`/api/v1`. Interactive docs at `/api/v1/docs` exist only in local/test.
Format	JSON in and out; file upload uses multipart (`POST /admin/resources`).
Authentication	HttpOnly `access_token` cookie (or `Authorization: Bearer`). See Part 5.
CSRF	Unsafe methods carrying the cookie must send `X-CSRF-Token` equal to the `csrf_token` cookie. Exempt: sign-in endpoints, refresh, webhooks, guardian links.
Idempotency	Payment application and webhooks are idempotent by provider ids; unpaid order reuse for 30 min.
Pagination	`limit` query parameter with a hard cap (500 admin lists, 200 attempts).
Tracing	Every response carries `x-request-id`; the same id is in logs and audit rows.
Caching	`Cache-Control: no-store` on all API responses except signed media (private, 1 h).

HTTP	code	Meaning for the UI
401	unauthenticated · token_expired	Client refreshes once, then asks to sign in
403	forbidden · not_entitled · guardian_consent_required · csrf_failed	Missing permission, subject not bought, minor awaiting consent, bad CSRF
404	not_found	Also used to hide resources that belong to someone else
409	conflict · already_registered · course_full · course_closed	State conflict
422	validation_failed · otp_invalid · otp_expired …	Field-level messages in `errors[]`
423	account_locked	Staff lock after 5 bad passwords
429	rate_limited	OTP request limit
502	gateway_unavailable	Payment gateway unreachable

3.2 Endpoint inventory

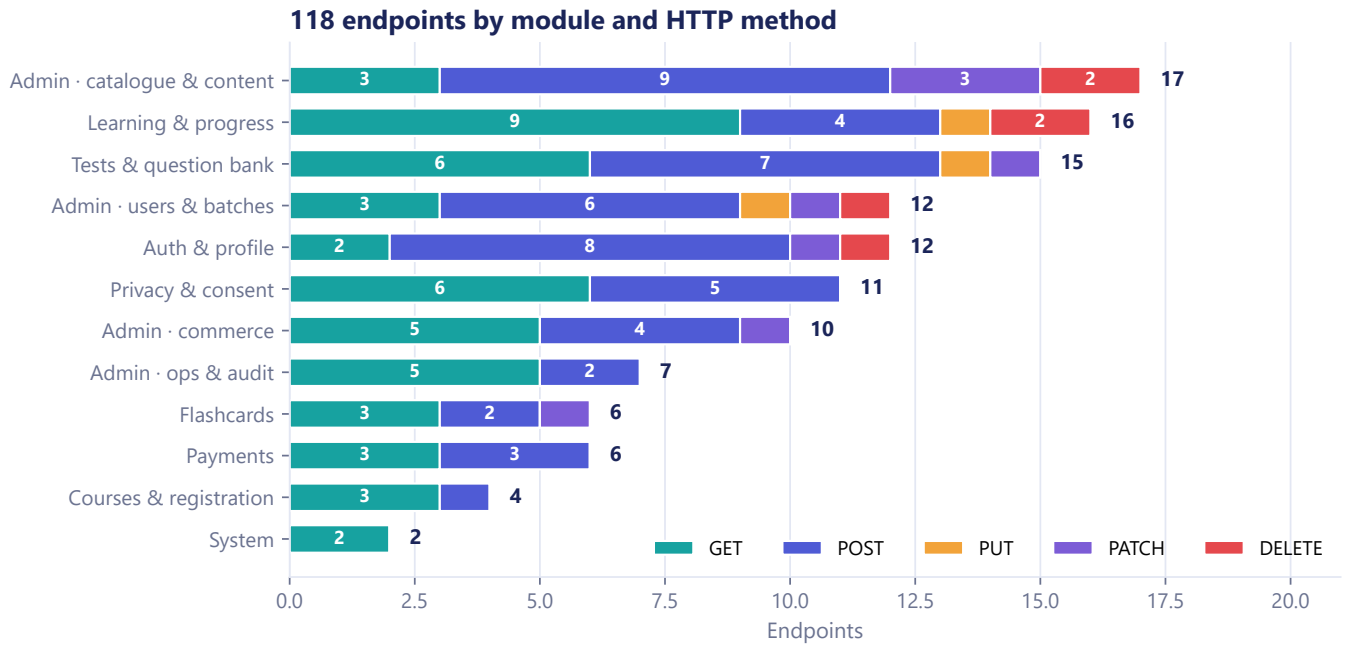
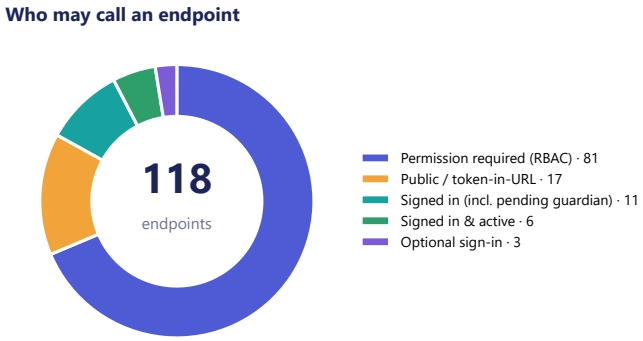
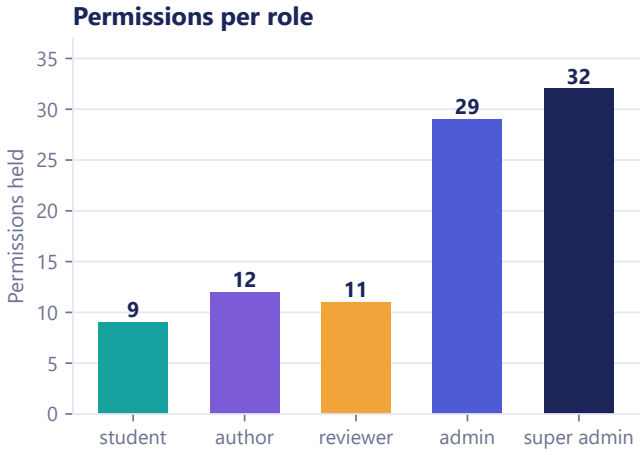


Figure 43. Endpoints per module, split by HTTP method.





> **Reading the access column**
> `public` no sign-in (some use a token in the URL or a signature). `signed in` any active user. `signed in*` also allowed while a minor awaits guardian consent. `optional` works signed out, richer when signed in. Permission names (e.g. `order:refund`) are checked against the caller's roles.

3.3 Endpoint catalogue

Auth & profile · 12 endpoints

Endpoint	Access	What it does
`POST` `/auth/otp/request`	public	Send a 6-digit code to a mobile or email
`POST` `/auth/otp/verify`	public	Check the code; sign in or return a signup token

Endpoint	Access	What it does
POST /auth/signup	public	Create the student account and start a session
POST /auth/staff/login	public	Staff email + password; returns MFA step
POST /auth/staff/mfa/setup	public	First-time TOTP secret and otpauth URI
POST /auth/staff/mfa	public	Verify TOTP and start a staff session
POST /auth/refresh	public	Rotate refresh token, issue new access token
POST /auth/logout	signed in*	End the current session
GET /me	signed in*	Current user with permission list
PATCH /me	signed in*	Update own profile
GET /me/sessions	signed in*	List signed-in devices
DELETE /me/sessions/{session_id}	signed in*	End one of my sessions

Privacy & consent · 11 endpoints

Endpoint	Access	What it does
GET /privacy/notice/current	public	Current privacy notice text and version
GET /me/consents	signed in*	My consent history
POST /me/consents	signed in	Grant or withdraw an optional consent
GET /me/guardian	signed in*	Guardian status for a minor
POST /me/guardian	signed in*	Send consent link to a guardian
GET /guardian-consent/{token}	public	Guardian views the notice
POST /guardian-consent/{token}/otp	public	Send OTP to the guardian's mobile
POST /guardian-consent/{token}	public	Guardian accepts with OTP; activates account
GET /me/data-requests	signed in*	My data-rights requests
POST /me/data-requests	signed in*	File access / correction / erasure / grievance / nomination
GET /me/data-export	signed in*	Download my data as JSON

Courses & registration · 4 endpoints

Endpoint	Access	What it does
GET /courses	public	Public course catalogue
GET /courses/{code}	optional	Course detail; shows registration state if signed in
POST /courses/{code}/register	course:register	Create registration and order, return checkout
GET /me/registrations	course:register	My registrations

Payments · 6 endpoints

Endpoint	Access	What it does
GET `/me/orders`	order:self	My orders and receipts
GET `/payments/orders/{order_id}`	order:self	One order (owner only)
POST `/payments/orders/{order_id}/confirm`	order:self	Verify checkout signature, fetch payment, apply
POST `/payments/webhooks/{provider}`	public	Signed gateway webhook (idempotent)
GET `/payments/mock/orders/{provider_order_id}`	public	Demo gateway: order info
POST `/payments/mock/complete`	order:self	Demo gateway: simulate pay / fail

Learning & progress · 16 endpoints

Endpoint	Access	What it does
GET `/subjects`	optional	Subjects with progress and an entitled flag
GET `/subjects/{code}`	optional	Subject with LMs
GET `/lms/{lm_id}`	signed in	LM detail: topics, items, resources, stats
GET `/content/{item_id}`	signed in	Published note body
GET `/resources/{rid}/playback`	signed in	Signed URL + watermark + resume position
GET `/media/stream/{token}`	public	Stream file by signed token (Range)
POST `/resources/{rid}/progress`	progress:write	Heartbeat: watched interval / position / complete
POST `/lms/{lm_id}/learned`	progress:write	Mark LM learned (24 h undo)
DELETE `/lms/{lm_id}/learned`	progress:write	Undo learned
POST `/lms/{lm_id}/revisions`	progress:write	Log a revision session
PUT `/lms/{lm_id}/eoc`	progress:write	Record end-of-chapter questions result
GET `/me/dashboard`	signed in	Home dashboard data
GET `/search`	signed in	Search LMs and notes
GET `/me/bookmarks`	bookmark:self	List bookmarks
POST `/me/bookmarks`	bookmark:self	Add bookmark
DELETE `/me/bookmarks/{bid}`	bookmark:self	Remove bookmark

Tests & question bank · 15 endpoints

Endpoint	Access	What it does
GET `/practice/options`	test:take	Subjects, chapters and counts for the quiz builder
POST `/tests/preview`	test:take	Pool size for chosen filters
POST `/tests`	test:take	Create custom test
POST `/lms/{lm_id}/quiz`	test:take	Start chapter quiz
POST `/subjects/{code}/test`	test:take	Start 50-question subject test

Endpoint	Access	What it does
GET `/attempts/{attempt_id}`	test:take	Load attempt (auto-submits if past deadline)
PUT `/attempts/{attempt_id}/items/{position}`	test:take	Save answer / flag / time
POST `/attempts/{attempt_id}/submit`	test:take	Submit and grade
GET `/me/attempts`	test:take	Attempt history
GET `/me/test-stats`	test:take	Accuracy by subject and difficulty
GET `/admin/questions`	catalog:read_admin	Filter question bank
POST `/admin/questions`	qbank:write	Create question
PATCH `/admin/questions/{qid}`	qbank:write	Edit question
POST `/admin/questions/{qid}/{action}`	qbank:publish	publish | retire | draft
GET `/admin/questions/validate`	catalog:read_admin	Release checks for an LM's question set

Flashcards · 6 endpoints

Endpoint	Access	What it does
GET `/flashcards/decks`	flashcard:review	Decks with due counts
GET `/flashcards/session`	flashcard:review	Cards due now
POST `/flashcards/{card_id}/review`	flashcard:review	Rate card, schedule next review
GET `/admin/flashcards`	catalog:read_admin	List flashcards
POST `/admin/flashcards`	flashcard:write	Create flashcard
PATCH `/admin/flashcards/{cid}`	flashcard:write	Edit / publish flashcard

Admin · catalogue & content · 17 endpoints

Endpoint	Access	What it does
GET `/admin/catalog`	catalog:read_admin	Full catalogue incl. drafts
POST `/admin/subjects`	catalog:write	Create subject
PATCH `/admin/subjects/{sid}`	catalog:write	Edit subject
POST `/admin/lms`	catalog:write	Create LM
GET `/admin/lms/{lm_id}`	catalog:read_admin	LM with items, resources, LOS
PATCH `/admin/lms/{lm_id}`	catalog:write	Edit LM
POST `/admin/topics`	catalog:write	Add topic
DELETE `/admin/topics/{tid}`	catalog:write	Remove topic
POST `/admin/los`	catalog:write	Add learning outcome
DELETE `/admin/los/{lid}`	catalog:write	Remove learning outcome
POST `/admin/content-items`	content:write	Create note item
GET `/admin/content-items/{iid}/versions/{vid}`	catalog:read_admin	Read a version
POST `/admin/content-items/{iid}/versions`	content:write	Save new draft version

Endpoint	Access	What it does
POST /admin/content-items/{iid}/versions/{vid}/publish	content:publish	Publish a version
POST /admin/resources	resource:write	Upload video / PDF / link (multipart)
PATCH /admin/resources/{rid}	resource:write	Edit resource metadata
POST /admin/resources/{rid}/{action}	resource:publish	publish | unpublish

Admin · commerce · 10 endpoints

Endpoint	Access	What it does
GET /admin/courses	course:manage	List courses
POST /admin/courses	course:manage	Create course
PATCH /admin/courses/{cid}	course:manage	Edit course, fee, status
GET /admin/registrations	order:read	Filter registrations
POST /admin/registrations/offline	order:offline	Enter an offline payment
POST /admin/registrations/{rid}/cancel	order:refund	Cancel and revoke access
GET /admin/orders	order:read	Filter orders
GET /admin/orders/{oid}	order:read	Order with payments and refunds
POST /admin/orders/{oid}/refund	order:refund	Full or partial refund
GET /admin/revenue	order:read	Revenue summary

Admin · users, batches, entitlements · 12 endpoints

Endpoint	Access	What it does
GET /admin/users	user:read	Search users
GET /admin/users/{uid}	user:read	User with roles, entitlements, sessions
POST /admin/users	user:write	Create staff user
PUT /admin/users/{uid}/roles	user:write	Set roles
POST /admin/users/{uid}/deactivate	user:write	Deactivate and end sessions
POST /admin/users/{uid}/reactivate	user:write	Reactivate
DELETE /admin/users/{uid}/sessions	session:revoke	Force sign-out everywhere
POST /admin/entitlements	entitlement:grant	Grant subject access manually
POST /admin/entitlements/{eid}/revoke	entitlement:grant	Revoke an entitlement
GET /admin/batches	batch:manage	List batches
POST /admin/batches	batch:manage	Create batch
PATCH /admin/batches/{bid}	batch:manage	Edit batch

Admin · reports, audit, data requests · 7 endpoints

Endpoint	Access	What it does
GET /admin/overview	report:read	KPI tiles

Endpoint	Access	What it does
GET /admin/reports/progress	report:read	Per-student progress for a course
GET /admin/audit	audit:read	Search the audit log
GET /admin/audit/verify	audit:read	Recompute the hash chain
GET /admin/data-requests	data_request:process	Privacy request queue
POST /admin/data-requests/{did}	data_request:process	Update status / resolution
POST /admin/data-requests/{did}/erase	data_request:process	Execute erasure

System · 2 endpoints

Endpoint	Access	What it does
GET /healthz	public	Liveness
GET /readyz	public	Readiness (database ping)

3.4 Database design

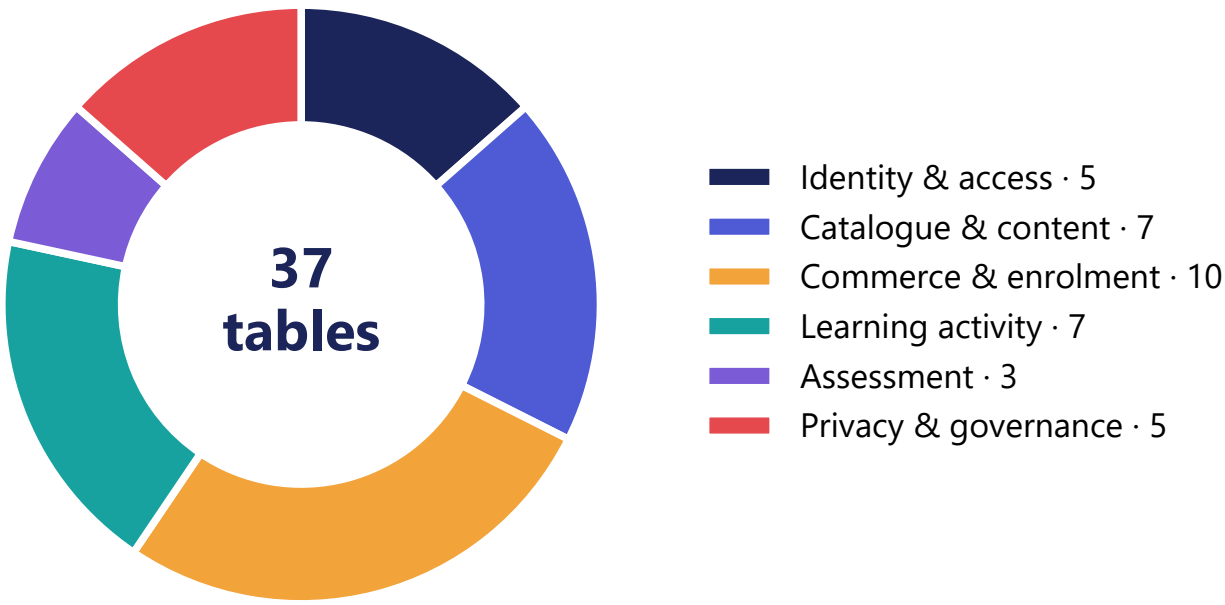
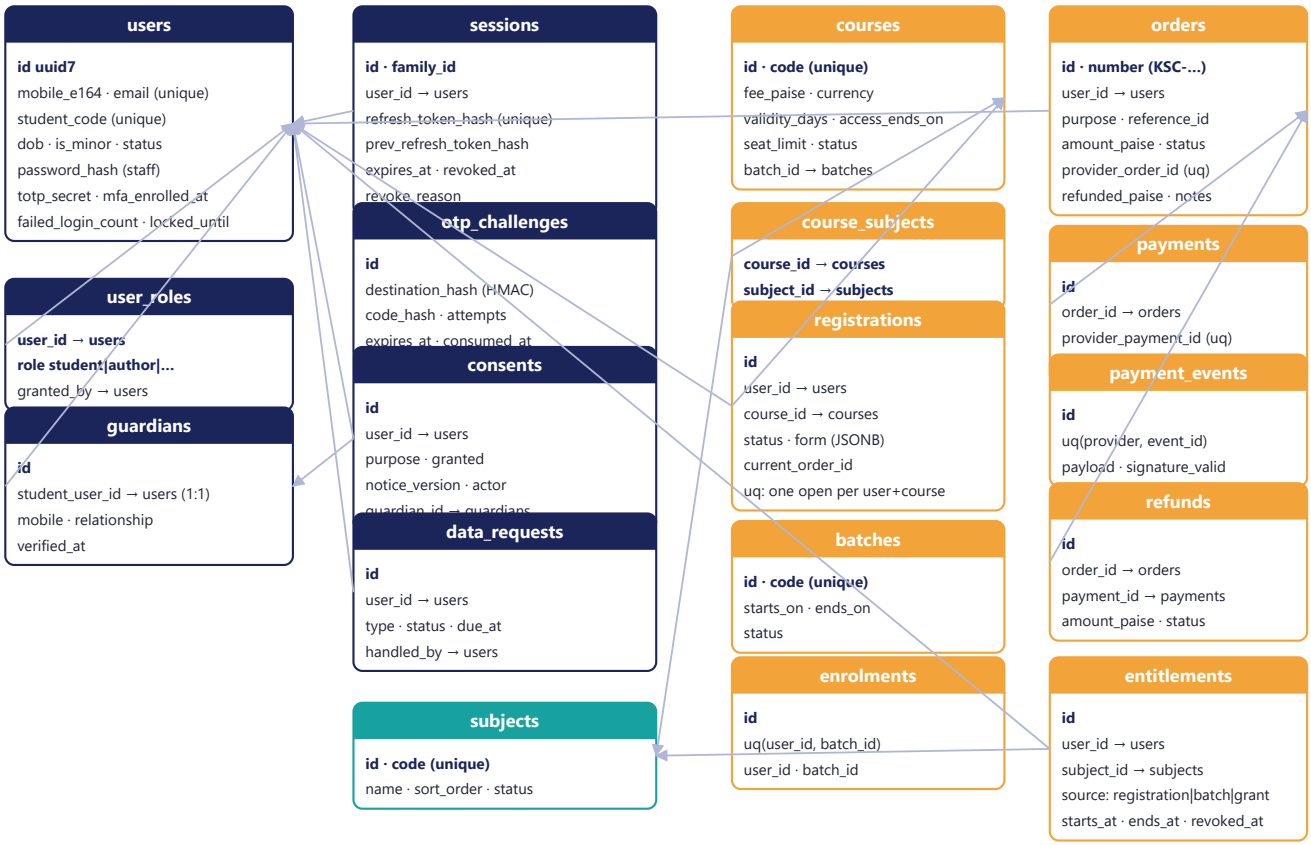


Figure 44. 37 tables grouped into six domains.



Colour: navy = identity & privacy · amber = commerce & enrolment · teal = catalogue. Arrows point from the referencing table to the referenced table.

Figure 45. Entity relationships · identity, privacy, commerce and enrolment (key columns only).



All user-owned rows reference users(id) with ON DELETE CASCADE; only orders and courses are RESTRICT so financial history survives. Colours: teal = catalogue · indigo = content · violet = assessment · gre

Figure 46. Entity relationships · catalogue, content, assessment and learning progress.

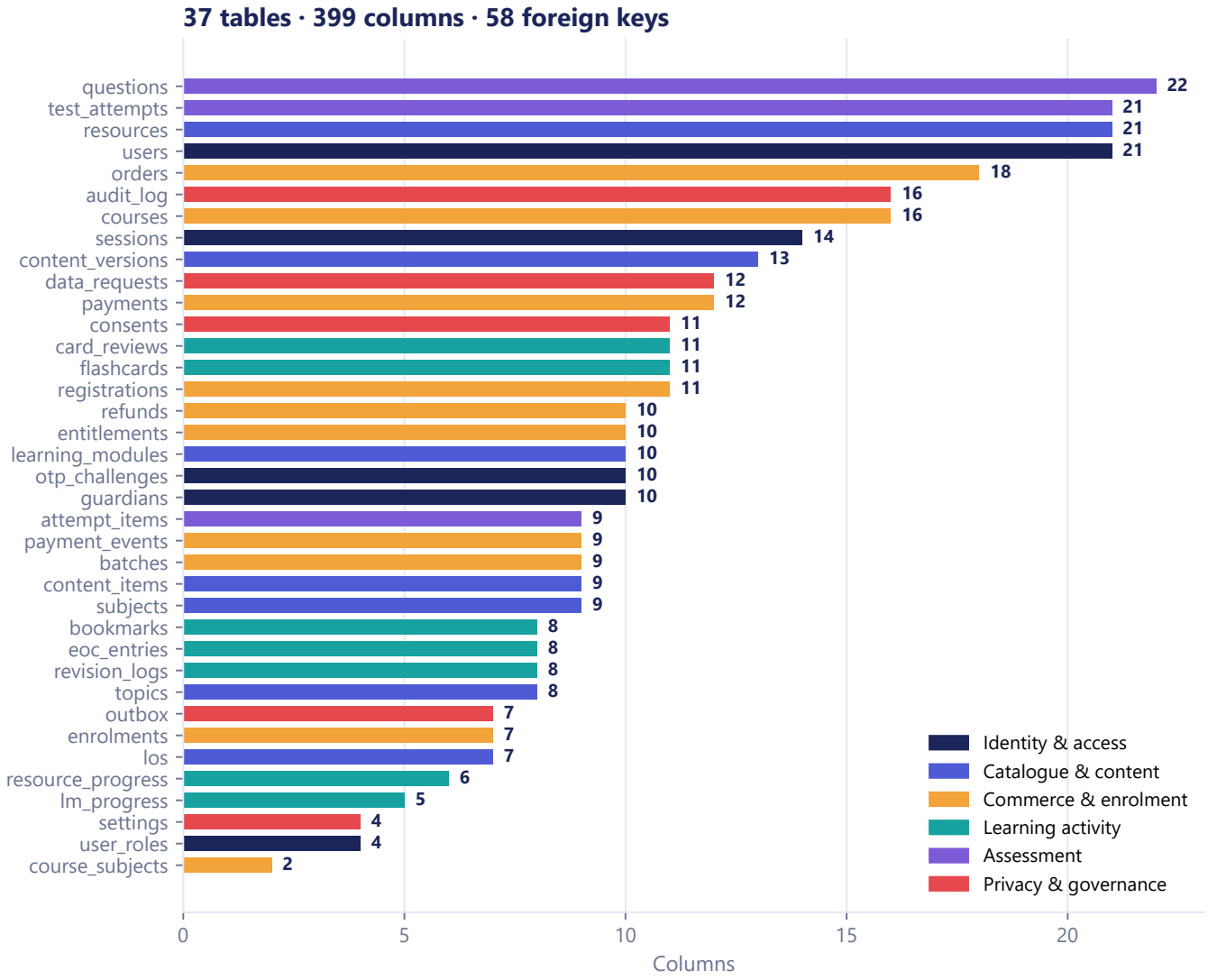


Figure 47. Width of each table (columns), coloured by domain.

Design rules

- **UUIDv7 primary keys** everywhere except two append-only logs (audit_log, outbox) which use identity integers to give a total order.
- **JSONB** only for genuinely flexible payloads: note bodies, question options, attempt snapshots, watched intervals, audit changes.
- **Referential actions**: user-owned rows CASCADE on delete (erasure is simple); orders and courses are RESTRICT so financial history cannot be orphaned.
- **Money** is stored as integer paise, never floats. GST-inclusive fee on the course.
- **Secrets** are stored only as hashes (OTP, refresh token, password) or, for TOTP, as base32 bytes awaiting KMS envelope encryption.
- **Constraints do the policing**: partial unique index for one open registration per user and course; unique (provider, event_id) for webhook replay; unique per-day revision count.

Data dictionary

Identity & access · 5 tables

Table	Purpose	Key columns & constraints	Notes
`users`	Every person: candidate or staff	mobile_e164 / email unique · status · is_minor · password_hash · totp_secret	CITEXT email; soft delete via deleted_at
`user_roles`	Role grants (composite PK)	user_id + role	CASCADE with user
`guardians`	Parent / guardian of a minor	student_user_id unique · verified_at	1 per student
`sessions`	One row per signed-in device	refresh_token_hash · family_id · revoked_at	hash only, never the token
`otp_challenges`	One-time codes	destination_hash · code_hash · attempts	hashes only

Catalogue & content · 7 tables

Table	Purpose	Key columns & constraints	Notes
`subjects`	10 CFA subjects	code unique · exam_weight	
`learning_modules`	56 LMs	unique(subject_id, lm_no) · source_ref (internal)	source_ref never exposed to candidates
`topics`	Sections inside an LM	los_ids JSONB	
`los`	Learning outcome statements	unique(lm_id, code) · command_word	
`content_items`	A note slot in an LM	current_published_version_id	
`content_versions`	Immutable versions of a note	unique(item_id, version_no) · body JSONB · plain_text (search)	provenance + licence_ref
`resources`	Video, PDF, class note, link	storage_key · chapters JSONB · downloadable	status draft→published

Commerce & enrolment · 10 tables

Table	Purpose	Key columns & constraints	Notes
`courses`	Sellable course	fee_paise (GST-inclusive) · validity_days or access_ends_on · seat_limit	status draft|open|closed
`course_subjects`	Which subjects a course includes	composite PK	
`registrations`	A candidate's seat in a course	partial unique index: one open registration per user+course	RESTRICT on course
`orders`	Money request	number KSC-year-seq · provider_order_id unique · refunded_paise	RESTRICT on user
`payments`	Gateway payment attempts	provider_payment_id unique	
`payment_events`	Raw webhooks for replay & idempotency	unique(provider, event_id)	signature_valid recorded
`refunds`	Refund records	amount_paise · status	
`batches`	Cohorts	code unique · starts_on / ends_on	
`enrolments`	User ↔ batch	unique(user_id, batch_id)	
`entitlements`	The access ledger: who may open which subject, until when	index(user, subject, ends_at) · source registration|batch|grant	single source of truth for access

Learning activity · 7 tables

Table	Purpose	Key columns & constraints	Notes
`lm_progress`	Learned flag and last opened	learned_undo_until	
`resource_progress`	Watched intervals per resource	watched JSONB merged intervals	

Table	Purpose	Key columns & constraints	Notes
`revision_logs`	Revision sessions	unique per user+LM+day when counted	
`eoc_entries`	End-of-chapter results	q_attempted · q_correct	
`bookmarks`	Saved places	target_type · target_id	
`flashcards`	Front/back cards	source_ref unique	
`card_reviews`	Per-user SRS state	ease · interval · due_on (indexed)	

Assessment · 3 tables

Table	Purpose	Key columns & constraints	Notes
`questions`	3-option MCQs	options JSONB · correct_index · difficulty · status draft|published|retired	version, provenance
`test_attempts`	A quiz or test run	kind chapter|master|custom · mode · deadline_at · score_pct	
`attempt_items`	Questions in an attempt	snapshot JSONB · answer_index · is_correct	snapshot freezes content

Privacy & governance · 5 tables

Table	Purpose	Key columns & constraints	Notes
`consents`	Append-only consent ledger	purpose · granted · notice_version · actor self|guardian	never updated in place
`data_requests`	DPDP rights requests	type · due_at · status	
`audit_log`	Tamper-evident audit trail	prev_hash · hash (SHA-256 chain) · changes/metadata JSONB	INET ip
`outbox`	Domain events awaiting dispatch	dispatched_at · attempts	
`settings`	Key/value config	JSONB value	

3.5 UI → API → database mapping

Read each row left to right: a screen calls the listed endpoints, which read or write the listed tables. Use it to estimate the blast radius of a change in either direction.

Screen (route)	API calls (under /api/v1)	Tables touched
`/login`	otp/request · otp/verify · signup · privacy/notice	otp_challenges, users, user_roles, sessions, consents, audit_log
`/pending-consent`	me/guardian	guardians, consents
`/guardian-consent/[token]`	guardian-consent/{t} · …/otp	guardians, otp_challenges, consents, users
`/courses · /courses/[code]`	courses · courses/{code}	courses, course_subjects, registrations
`/register/[code]`	courses/{code}/register · payments/orders/{id}/confirm	registrations, orders, payments, users
`/mock-gateway (demo)`	payments/mock/* · payments/orders/{id}/confirm	orders, payments, payment_events
`/checkout/[orderId] · /orders`	payments/orders/{id} · me/orders	orders, payments, refunds

Screen (route)	API calls (under /api/v1)	Tables touched
`/dashboard`	me/dashboard · subjects	lm_progress, resource_progress, entitlements, test_attempts
`/learn · /learn/[subject]`	subjects · subjects/{code} · lms/{id} · content/{id} · resources/{id}/playback · progress · learned · revisions · eoc	subjects, learning_modules, topics, los, content_items/versions, resources, lm_progress, resource_progress, revision_logs, eoc_entries
`/practice`	practice/options · tests/preview · tests · subjects/{code}/test	questions, test_attempts, attempt_items, entitlements
`/test/[id]`	attempts/{id} · items/{n} · submit · lms/{id}/quiz	test_attempts, attempt_items, questions, audit_log
`/flashcards`	flashcards/decks · session · {id}/review	flashcards, card_reviews
`/bookmarks · /search`	me/bookmarks · search	bookmarks, content_items/versions, learning_modules
`/profile`	me · me/sessions · me/consents · me/data-requests · me/data-export	users, sessions, consents, data_requests
`/staff/login`	auth/staff/login · mfa/setup · mfa	users, sessions, audit_log
`/admin (overview)`	admin/overview	users, registrations, orders, data_requests
`/admin/catalog · /admin/lms/[id]`	admin/catalog · lms · topics · los · content-items · resources	subjects, learning_modules, topics, los, content_*, resources
`/admin/questions`	admin/questions · …/{id}/{action} · validate	questions
`/admin/flashcards`	admin/flashcards	flashcards
`/admin/courses`	admin/courses	courses, course_subjects, batches
`/admin/registrations`	admin/registrations · offline · cancel	registrations, orders, entitlements
`/admin/orders · /admin/orders/[id]`	admin/orders · refund · revenue	orders, payments, refunds
`/admin/users · /admin/users/[id]`	admin/users · roles · deactivate · sessions · entitlements	users, user_roles, sessions, entitlements
`/admin/batches`	admin/batches	batches, enrolments
`/admin/reports`	admin/reports/progress	lm_progress, test_attempts, registrations
`/admin/data-requests`	admin/data-requests · erase	data_requests, users and all user-owned rows
`/admin/audit`	admin/audit · verify	audit_log

Flow: taking a quiz, from click to stored result

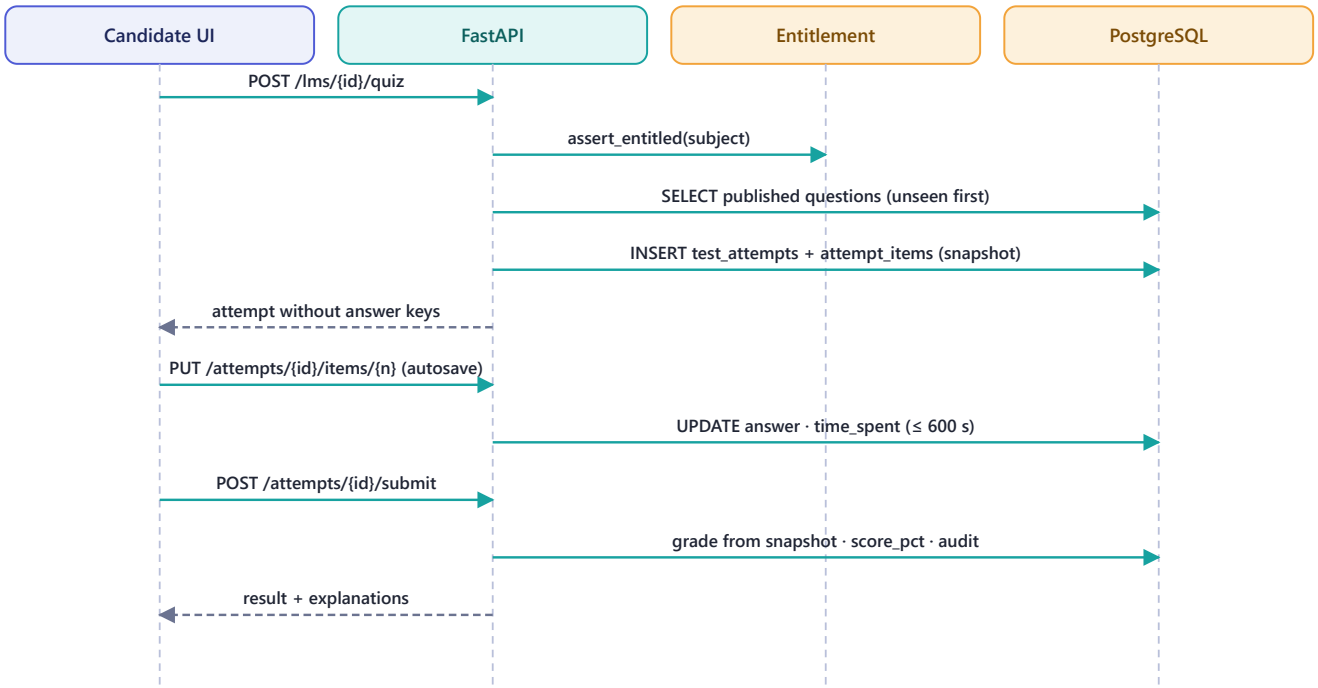


Figure 48. Quiz data flow. The answer key never leaves the server before submission; the result is graded from the stored snapshot.

PART 4

Scaling & performance

Workload assumptions, objectives, how the code scales today, known limits and the roadmap.

Targets, models and measurements
No load test has been run against this build yet (the plan, N-04, is documented). Everything in this part is therefore one of: a **requirement** from the NFR specification (N-01), a **design** from the infrastructure document (A-05), a **mathematical model** built from those assumptions and from real settings in the code, or a **fact** read from the code. Each is labelled.

4.1 Workload assumptions

Parameter	Value	Kind
Registered students	2,000 in year 1; design headroom to 10,000	Requirement (N-01)
Daily active students	600	Requirement
Peak concurrent users	300 (evening 20:00–23:00 IST, batch start); 500 at contest start	Requirement
Peak API request rate	≈150 req/s at 300 users (≈0.5 req/s per user, heartbeats dominate)	Requirement
Content	56 LMs, ≈200 h video, ≈1,000 PDFs/notes, 5,000 questions by end of phase 2	Requirement
Data growth	≈5 GB / year relational	Requirement
Required headroom	3× peak = 900 users ≈ 450 req/s for 30 minutes with latency SLO met	Requirement (N-01 §4)

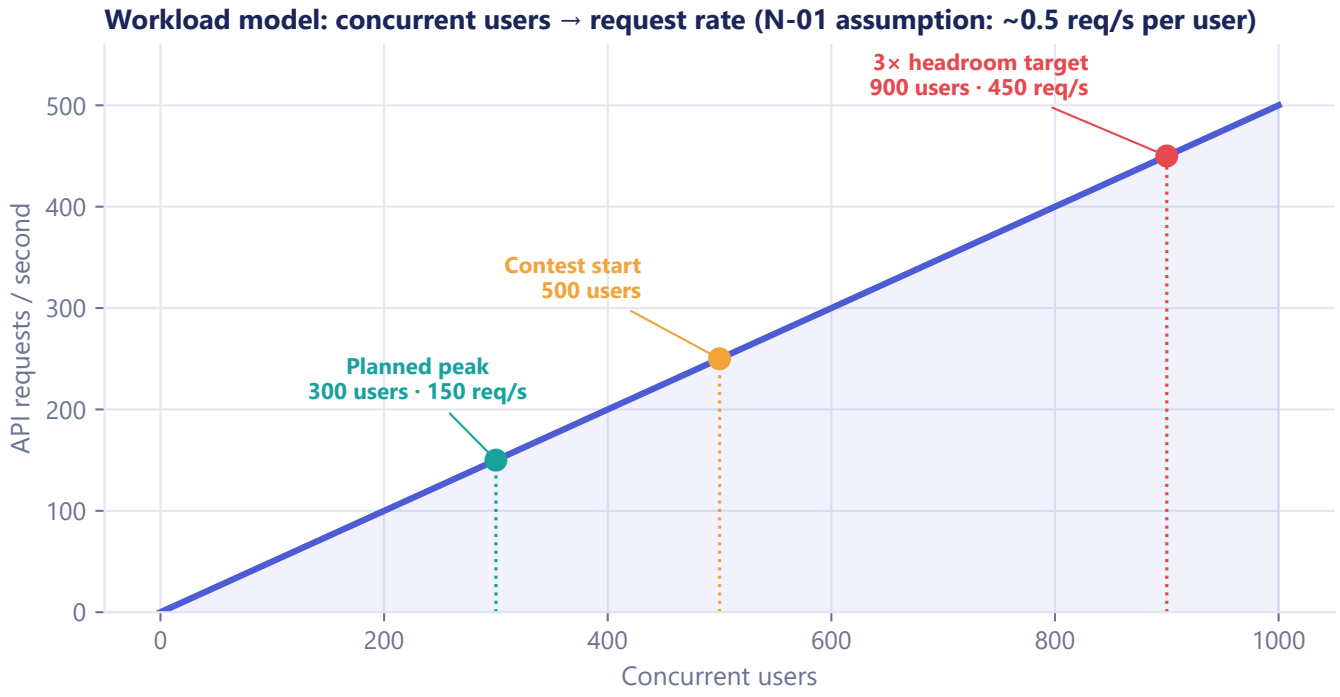


Figure 49. Workload model derived from the N-01 assumption. The three marked points are the planning milestones.

4.2 Service-level objectives

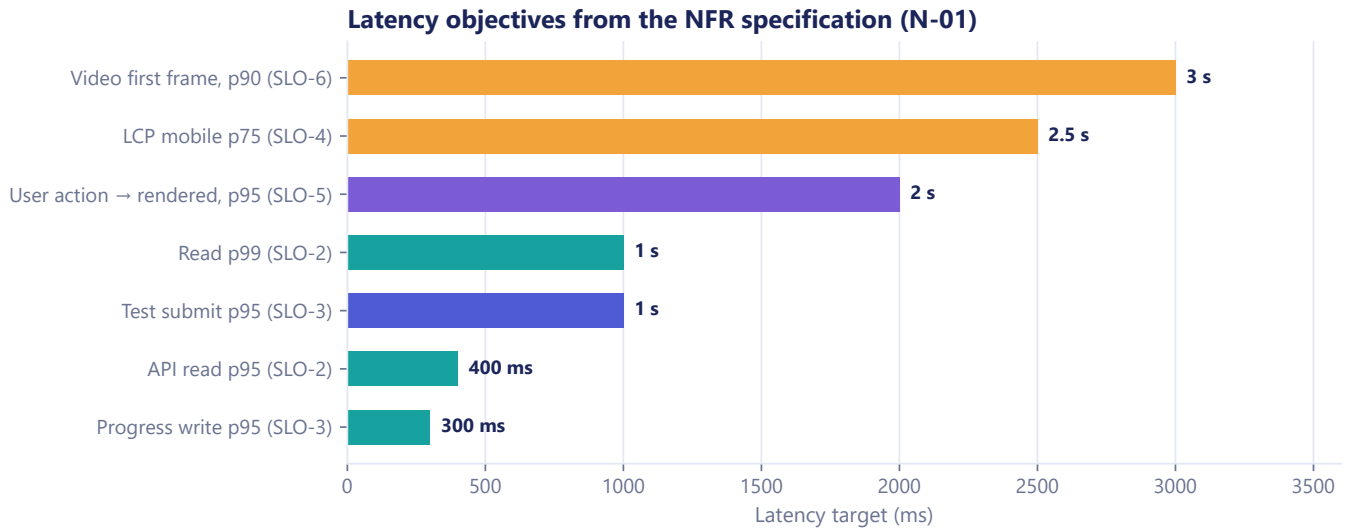
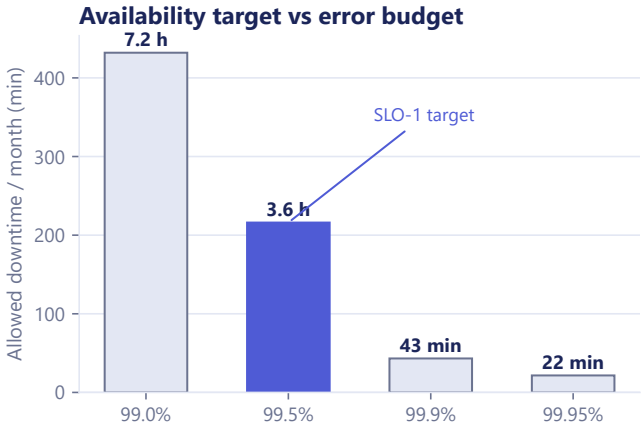


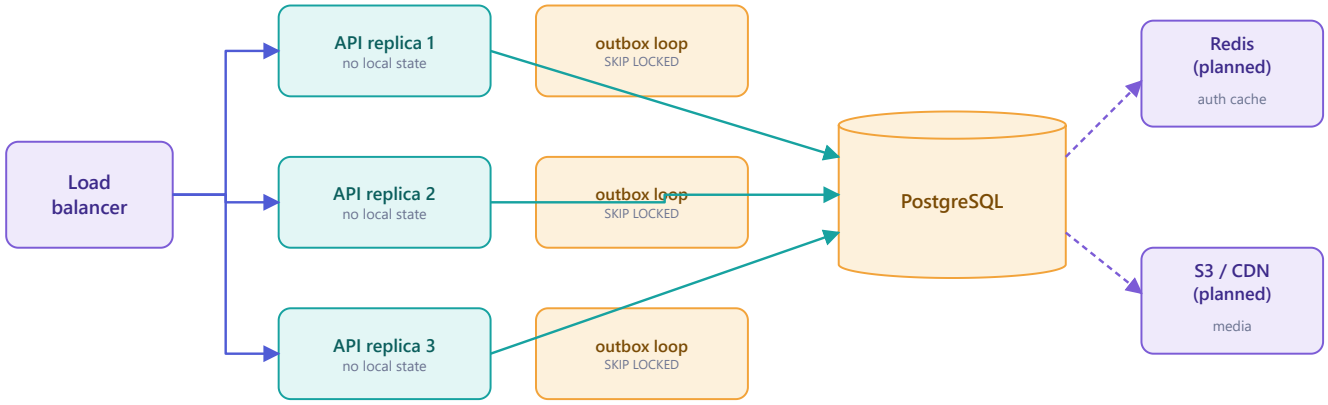
Figure 50. Latency objectives (requirements).

SLO	Target
SLO-1 Availability	99.5% monthly (≈3 h 39 min budget)
SLO-2 API read latency	p95 ≤ 400 ms · p99 ≤ 1 s
SLO-3 Writes	progress p95 ≤ 300 ms · submit ≤ 1 s
SLO-4 Page experience	LCP p75 ≤ 2.5 s · INP ≤ 200 ms · CLS ≤ 0.1
SLO-5 End-to-end action	p95 ≤ 2 s
SLO-6 Video start	p90 ≤ 3 s
SLO-7 Durability	no acknowledged write lost
SLO-8 OTP delivery	≥ 97% within 30 s



Reliability item	Target
RPO / RTO (AZ failure)	≈ 0 / ≤ 5 min (Multi-AZ, automatic)
RPO / RTO (region)	≤ 15 min / ≤ 4 h (runbook)
Backups	PITR 14 days; daily snapshots 35 days; cross-region copy; quarterly restore test
Deploys	Zero-downtime rolling; expand/contract migrations

4.3 How the code is built to scale



Any replica can serve any request: identity lives in the database-backed session row, not in memory.

Figure 51. Stateless API tier. Replicas share nothing in memory, so adding capacity is a configuration change.

Technique	Fact in the code	Effect
Stateless requests	Sessions are rows in PostgreSQL; the JWT carries only ids	Any replica serves any request; no sticky sessions
Async I/O	FastAPI + asyncpg + async SQLAlchemy	One worker holds many in-flight requests while waiting on the database
Connection pooling	`pool_size=10` (+10 default overflow), `pool_pre_ping`	Bounded DB concurrency per process; stale connections are replaced
One transaction per request	`get_session()` commits once or rolls back	Short, predictable lock windows
Cheap list endpoints	Hard `limit` caps; LM topics and course subjects use `selectin` eager loading	Avoids N+1 queries (budget ≤ 10 queries per request)
Indexes on hot paths	user_id on every user-owned table; due_on on card_reviews; status on questions and attempts; (user, subject, ends_at) on entitlements	Dashboard, quiz draw and access checks stay index-driven
Idempotent writes	Webhook unique key; payment applied once; revision once per day	Retries and duplicate clicks are harmless
Multi-replica background work	Outbox dispatch uses `SKIP LOCKED`	No leader election needed
Range-capable media	`FileResponse` with browser Range seeking and 1-hour private cache	Seeking in video without re-download; moves behind a CDN later

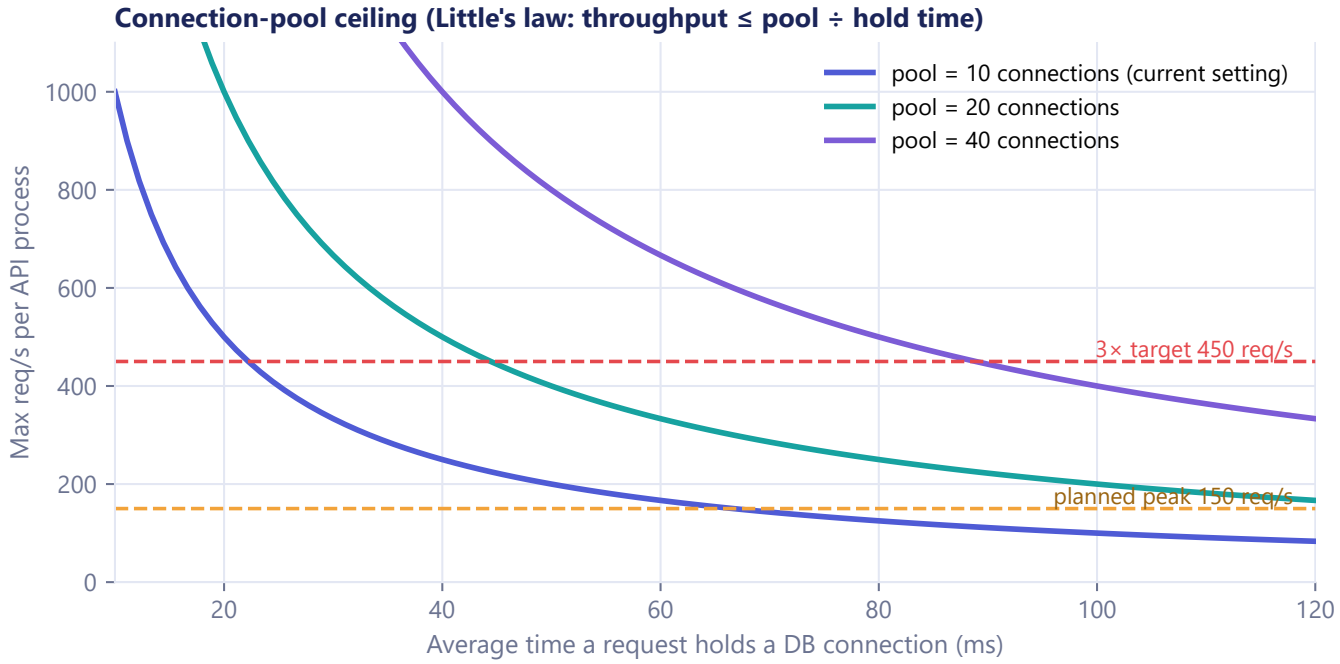
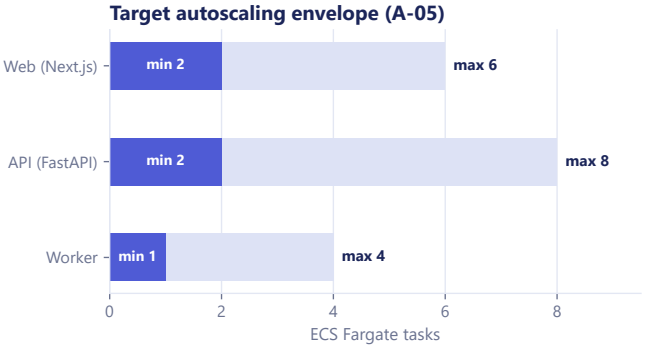


Figure 52. Model: with a pool of 10 connections and ~40 ms of connection time per request, one API process tops out near 250 req/s, comfortably above the 150 req/s planning peak but below the 450 req/s headroom goal, which is why two or more replicas are planned.



Tier	Today	Target scaling rule
Web	1 process	2–6 tasks · CPU 60%
API	1 process	2–8 tasks · CPU 60% + requests per target
Worker	in-process loop	1–4 · queue depth
Database	Render PG	t4g.medium Multi-AZ → xlarge → read replica at CPU > 60%
Cache	none	Redis primary + replica

4.4 Known limits and bottlenecks

Honest list of what would limit growth, ordered by when it would bite.

#	Area	Today	Risk	Planned remedy
1	Media on local disk	Files stored under `backend/var/storage`	Ties the API to one machine; lost on redeploy on Render free; heavy video traffic competes with API CPU	S3 + CloudFront signed URLs (A-05); storage interface already abstracts this
2	Auth lookups	Each request loads the session row and the user row	2 extra queries per call at 150–450 req/s	Short-TTL Redis cache keyed by session id; revocation clears the key
3	Global audit lock	`pg_advisory_xact_lock` serialises every audit write to keep the hash chain linear	Throughput ceiling for audited actions (logins, orders, publishing); not hit by reads or autosave	Acceptable at launch volumes; shard the chain per day or per actor if audited writes exceed hundreds/s
4	Progress heartbeats	Watched intervals merged in JSONB on each ping	Write amplification at 300 concurrent viewers	Client-side batching (send every 15–30 s); already within the 20 req/s heartbeat budget

#	Area	Today	Risk	Planned remedy
5	Search	ILIKE on titles plus `to_tsvector` computed at query time; pg_trgm installed, no GIN index yet	Sequential scans as notes grow into the thousands	Add GIN index on a stored tsvector column and trigram index on titles
6	Reports	Progress report computed synchronously	Slow query at 10,000 students	Run as job or on read replica (N-03)
7	Rate limiting	Per-destination OTP limit only	No generic per-IP or per-user throttle inside the app	WAF rate rules on `/api/v1/auth/*`; add middleware limiter backed by Redis
8	Question draw	Eligible pool loaded in Python then sampled	O(pool) per quiz start; fine for 5,000 questions	Push sampling into SQL if the bank grows 10×
9	Single instance	One container on Render	No redundancy; cold start on free plan	Stage 1–2 of the roadmap

4.5 Scaling roadmap

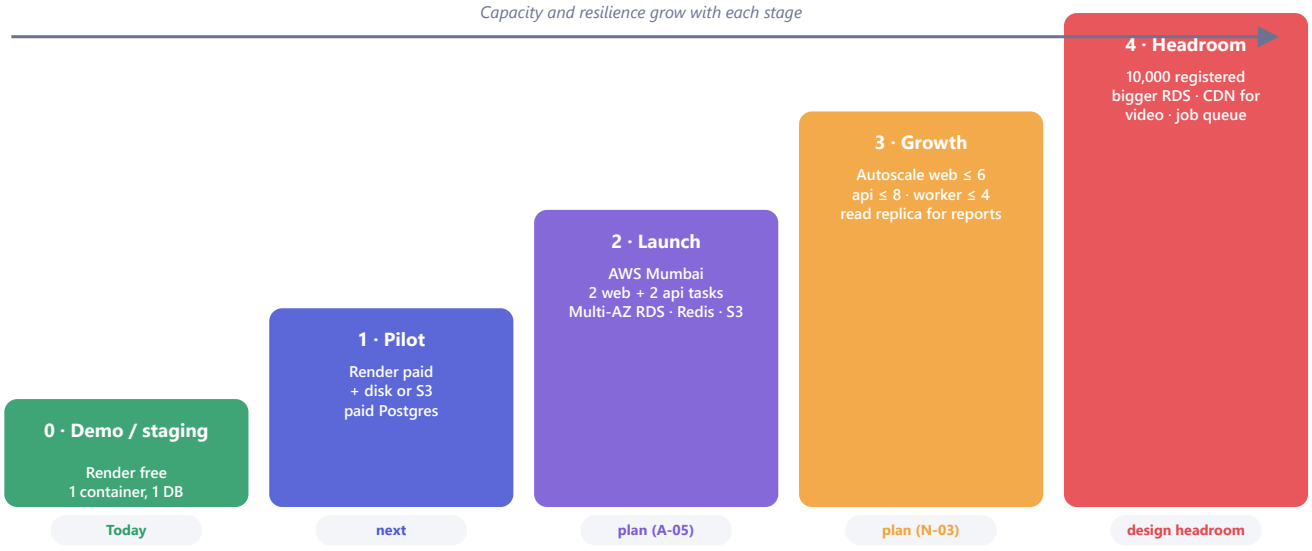


Figure 53. Staged path from demo to headroom for 10,000 registered students. Stages 2–4 are planned designs.

4.6 Performance budgets and verification

Budget (N-01 §3)	Limit	How it is checked
JS shipped on student routes (gzip, first load)	≤ 180 KB	Lighthouse CI on pull requests (planned)
Fonts	Figtree subset WOFF2 ≤ 120 KB	Build check
DB queries per API request	≤ 10	N+1 detector in tests with SQLAlchemy event counters (planned)
DB statement timeout (API role)	5 s	Role setting; long reports run as jobs
API payload	≤ 200 KB typical; lists ≤ 200 items	Schema review
Load test	3× peak for 30 min with SLO-2 met	N-04 plan, not yet executed

PART 5

Authentication & security

Every credential, token and control that protects candidates, staff and money.

5.1 Defence in depth

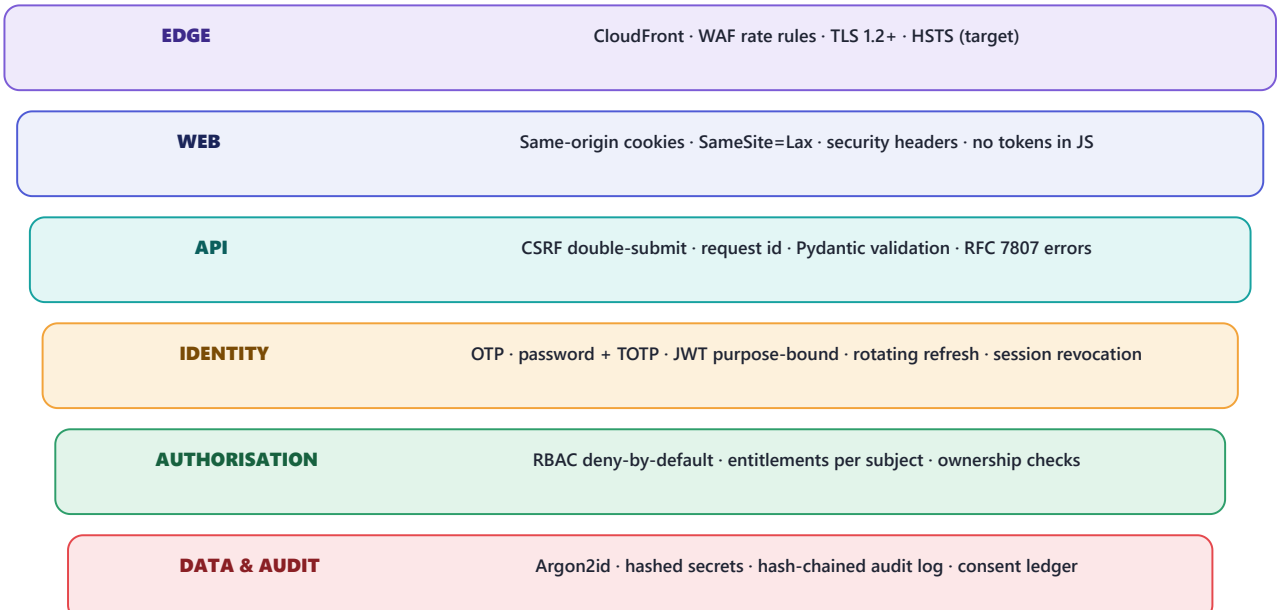


Figure 54. Six layers. A request must pass each one; a failure at any layer stops it.

5.2 Catalogue of authentication mechanisms

#	Mechanism	Used by	How it works	Key parameters
1	Mobile OTP	Candidates	6-digit random code; stored as sha256(challenge_id:code); destination stored only as a keyed HMAC	5 min TTL · 5 requests/h · 5 guesses · single use
2	Email OTP	Candidates (API-ready)	Same engine, channel = email	same
3	Password + TOTP	Staff	Argon2id password, then RFC 6238 authenticator code (±1 step window)	5 fails → 15 min lock · TOTP mandatory outside dev
4	Signup token	New candidates	JWT `purpose=signup` proves the verified mobile until the form is submitted	20 min
5	MFA step-up token	Staff	JWT `purpose=mfa` carries the user between password and TOTP	5 min
6	Access token	Everyone signed in	HS256 JWT in HttpOnly cookie `access_token`; carries sub, session id, roles; session row re-checked on every call	15 min
7	Refresh token	Everyone signed in	Opaque random token, only its sha256 stored; rotated on every use; reuse of an old one revokes the session	Student 30 d · staff 12 h
8	CSRF token	Browser writes	Double-submit: cookie `csrf_token` must equal header `X-CSRF-Token`	per session
9	Guardian consent	Parents of minors	JWT link (`purpose=guardian`) plus the guardian's own OTP	link 7 d · OTP 5 min
10	Signed media URL	Video / PDF viewers	JWT `purpose=media` bound to resource and user	2 h
11	Checkout signature	Payments	Gateway signature verified server-side, then the payment is fetched from the gateway	per payment

#	Mechanism	Used by	How it works	Key parameters
12	Webhook HMAC	Gateway → API	Razorpay signature on raw body; event id deduplicated	per event
13	Event-sink HMAC	API → partner service	`x-event-signature` = HMAC-SHA256 of body with the secret key	per event

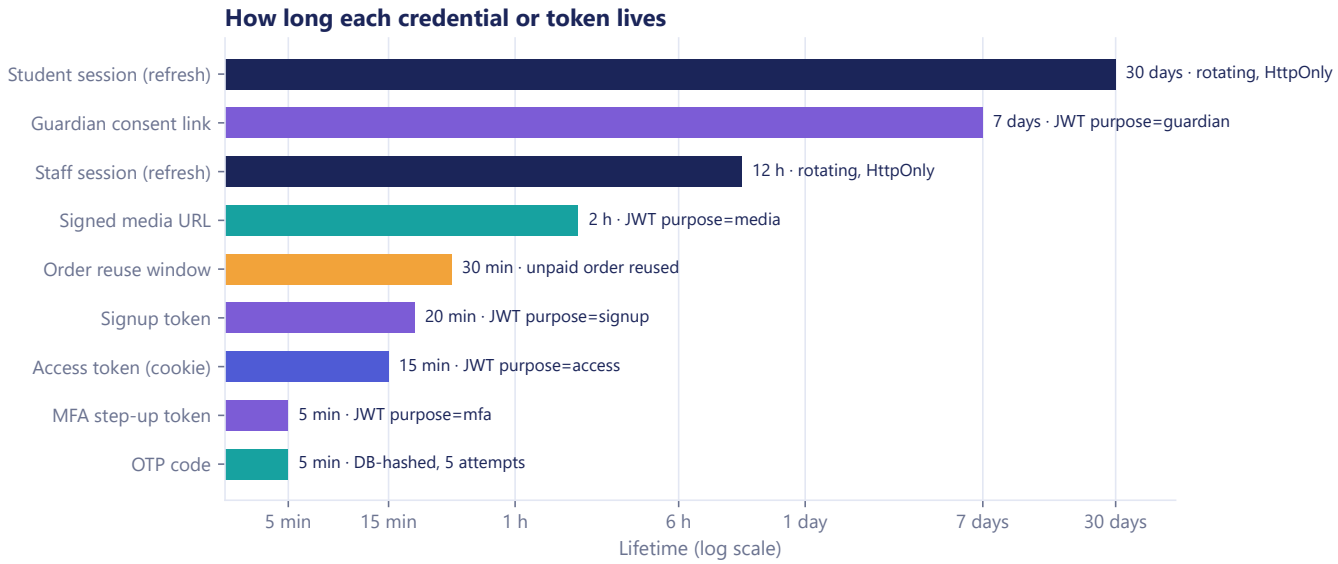


Figure 55. Lifetimes of every token and credential, from the code's configuration.

> ★ **Purpose-bound tokens**
> All JWTs share one signing key but carry a `pur` claim (access, signup, mfa, guardian, media). `decode_token(token, purpose)` rejects a token whose purpose differs, so a media link can never be used as an access token and vice versa.

5.3 Candidate sign-in sequence

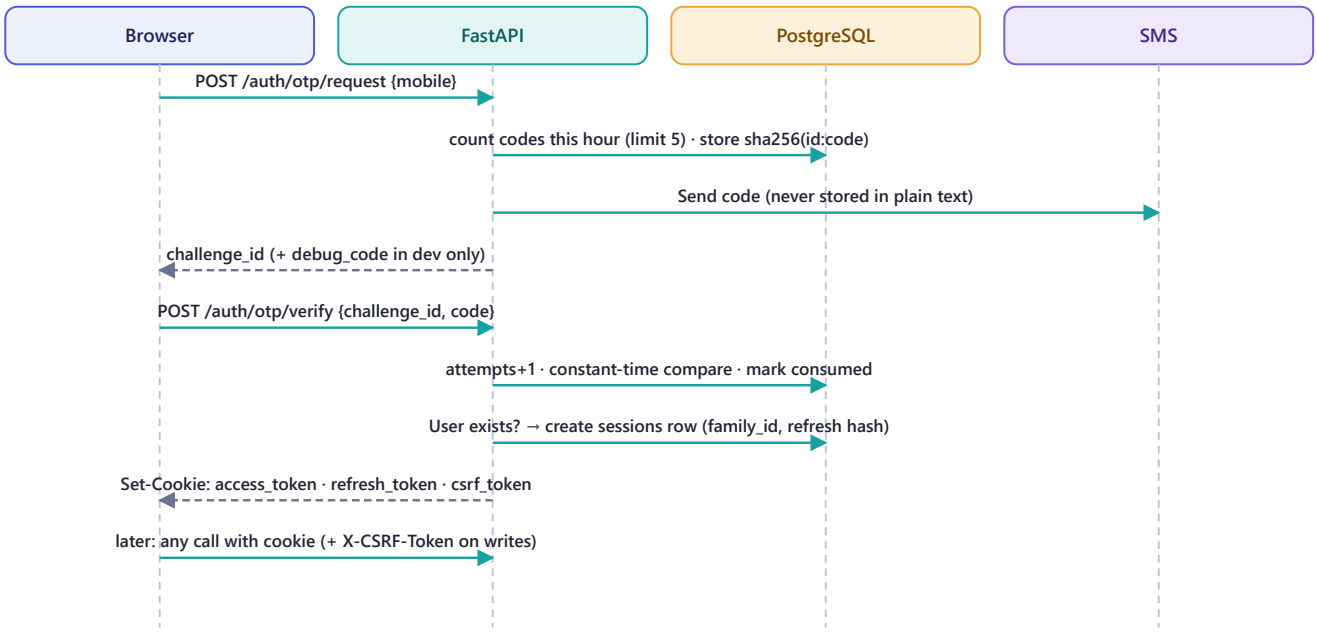


Figure 56. OTP sign-in from request to cookies.

5.4 Sessions, cookies and refresh rotation

Cookie	HttpOnly	Path	SameSite	Secure	Purpose
`access_token`	yes	/	Lax	required outside dev	Authenticates API calls
`refresh_token`	yes	/api/v1/auth	Lax	required outside dev	Only sent to the auth endpoints, so it is rarely exposed
`csrf_token`	no (read by JS)	/	Lax	required outside dev	Echoed in the X-CSRF-Token header

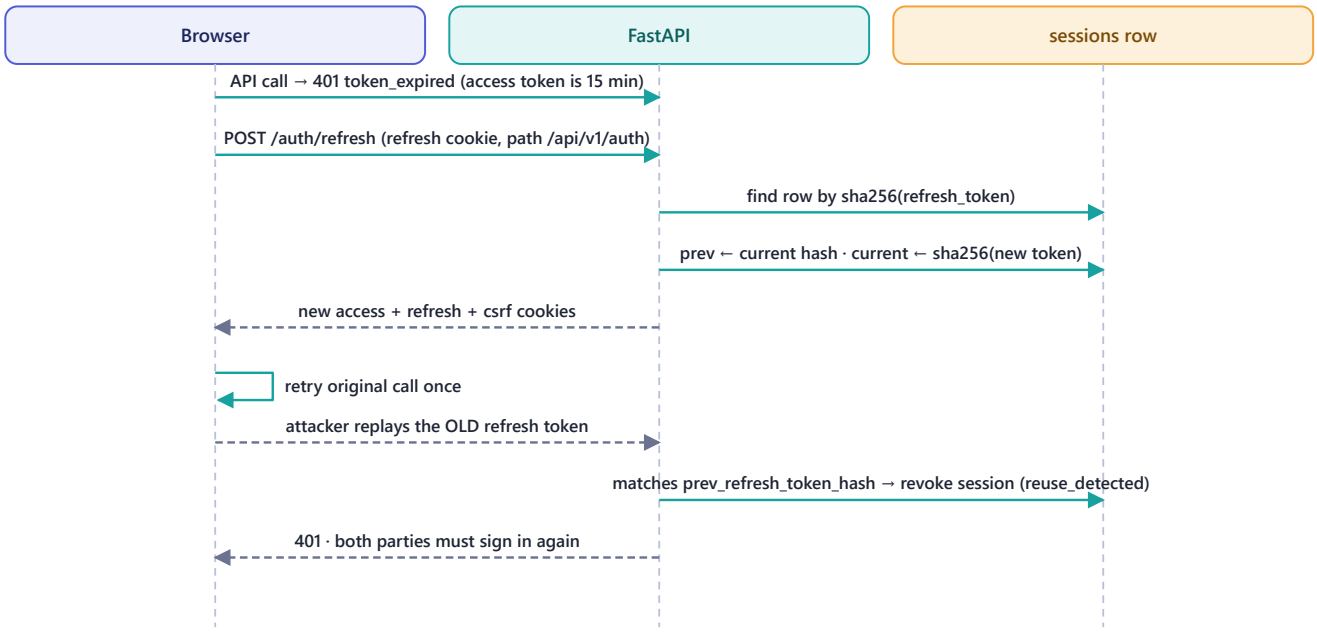
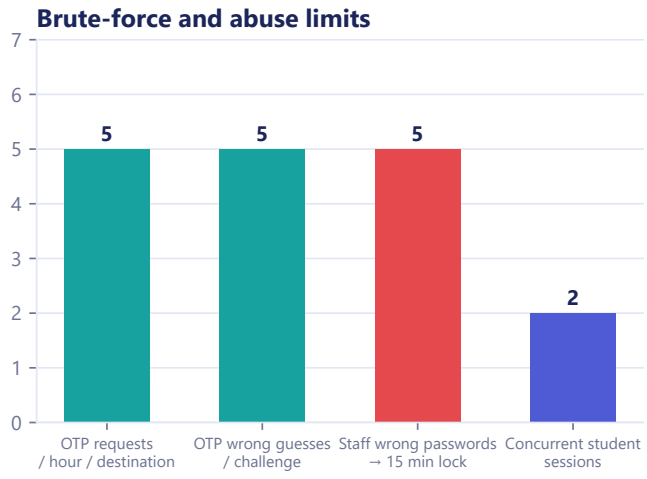


Figure 57. Rotating refresh tokens with reuse detection. The previous token's hash is kept so a replay is recognised and the whole session is killed.

- **Immediate revocation.** Because every request checks the `sessions` row, ending a session (logout, admin force sign-out, deactivation, erasure) takes effect on the very next call, not 15 minutes later.
- **Device limit.** Students may hold 2 live sessions; signing in on a third ends the oldest (BR-ACC-05). Staff are not limited but expire in 12 h.
- **Visible to the user.** Profile → Signed-in devices lists each session with device label and last seen.



5.5 Authorisation

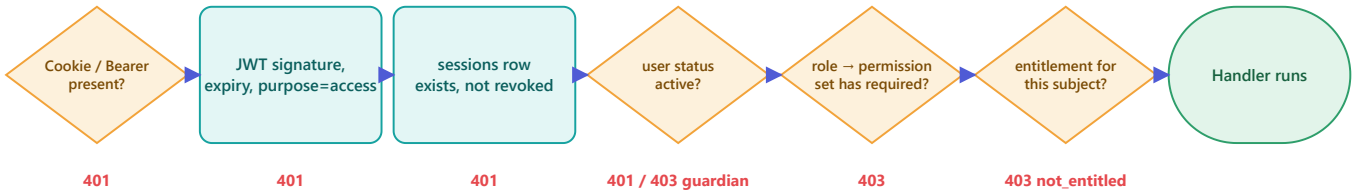


Figure 58. The order of checks for a protected API call. Red text is the response when that step fails.

- **RBAC, deny by default.** Roles map to permissions in one file; endpoints declare `require('perm')`. A role not listed grants nothing.
- **Entitlements** are the second gate for content: an LM, note, video, quiz or flashcard in a subject opens only when an active entitlement exists (granted by registration, batch or manual grant). Staff preview everything.
- **Ownership checks.** Attempts, orders, bookmarks, sessions and data exports are always filtered by the caller's user id; a wrong id returns 404, not 403, so ids cannot be probed.
- **Separation of duties.** Authors cannot publish; reviewers cannot edit commerce; only super admins assign admin roles.

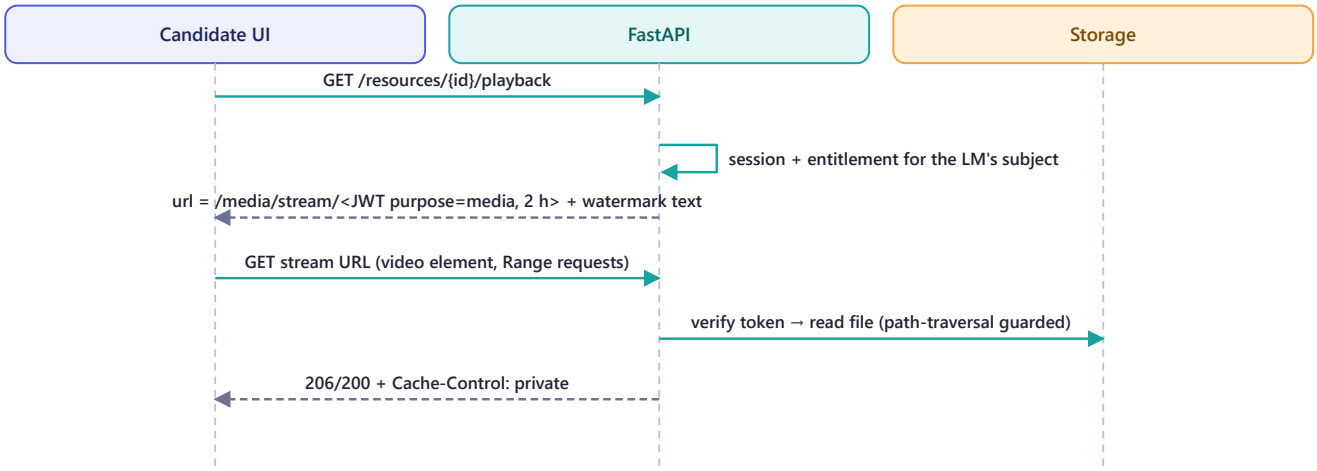


Figure 59. Protecting paid content: a short-lived signed URL, a visible watermark and no download unless allowed.

5.6 How credentials are stored

Secret	Stored as	Notes
Staff password	Argon2id · time 3 · memory 64 MiB · parallelism 2	argon2-cffi; verify is constant-time; generic error for unknown user and wrong password
OTP code	sha256(challenge_id : code)	Plain code exists only in the SMS
OTP destination	HMAC-SHA256 keyed with SECRET_KEY	Rate-limit counts without storing the number
Refresh token	sha256 of 256-bit random value	Database leak cannot be replayed
TOTP secret	base32 bytes in `users.totp_secret`	Never returned by the API; KMS envelope encryption is a TODO for the cloud move
JWT signing key	`SECRET_KEY` environment variable, ≥ 32 chars	Start-up refuses the default key outside dev

Secret	Stored as	Notes
Gateway secrets	Environment variables (Secrets Manager in target)	Never logged or stored in the database

5.7 Integrity, audit and privacy controls

- **Hash-chained audit log.** Each row stores `hash = SHA-256(previous hash + canonical row)`. `GET /admin/audit/verify` recomputes the chain; any edited or deleted row breaks every later hash. Writes are serialised with an advisory lock.
- **What is audited.** Sign-in and failures, lockouts, MFA enrolment, session revocation, user and role changes, orders, payments, refunds, mismatches, content and question publishing, resource uploads, consents, data requests and erasure.
- **Consent ledger.** Append-only rows with notice version, actor (self or guardian), method, IP and user agent.
- **Erasure.** Personal fields are overwritten and progress rows deleted; orders are kept as business records with the user pseudonymised.
- **Start-up guard rails.** Outside local/test the server refuses to run unless COOKIE_SECURE and STAFF_MFA_REQUIRED are on and SECRET_KEY is changed; DEMO_MODE and on-screen OTP are blocked in production.

5.8 Threats and the controls that answer them

Threat	Control in this build	Status
Password guessing (staff)	Argon2id, 5-fail lock for 15 min, MFA, generic errors, audit of failures	In place
OTP abuse / SMS bombing	5 codes/h per destination, 5 guesses, 5-min TTL; WAF rule planned	In place · edge rule planned
Token theft via XSS	HttpOnly cookies, short access token, nosniff, same-origin only	In place · CSP header not yet set
Cross-site request forgery	SameSite=Lax cookies plus double-submit CSRF header on writes	In place
Refresh-token replay	Rotation with previous-hash reuse detection	In place
Payment tampering	Server-side signature, gateway fetch, amount match, idempotent apply, webhook dedupe	In place
Content piracy	Entitlement check, 2-hour signed URLs, watermark, download off by default	In place · DRM not in scope
Insecure object references	Owner-scoped queries, UUIDv7 ids, 404 for foreign objects	In place
SQL injection	SQLAlchemy parameterised queries only	In place
Malicious upload	MIME allow-list, PDF signature check, size cap, safe file names	Partial · antivirus scan pending
Insider tampering	Hash-chained audit with verify endpoint, separation of duties	In place
Minor's data	Guardian consent gate before registration and learning	In place

5.9 Gaps and hardening backlog

These are visible in the code today and should be closed before real students and live payments.

Priority	Gap	Why it matters	Fix
High	SMS and email adapters are console-only (`NotImplementedError` for MSG91 / SES)	No real OTP can be delivered; demo mode shows codes on screen	Implement MSG91 and SES adapters, then set OTP_DEBUG_ECHO=false and DEMO_MODE=false

Priority	Gap	Why it matters	Fix
High	Razorpay keys and webhook not yet connected	Payments run on the mock gateway	Set PAYMENT_PROVIDER=razorpay, three keys, register the webhook URL
High	TOTP secrets not envelope-encrypted	Database leak would expose second-factor seeds	KMS-encrypt at rest (documented TODO)
Medium	No Content-Security-Policy header; HSTS expected from the edge	Lower XSS defence in depth	Add CSP in Next.js headers and HSTS at CloudFront
Medium	TOTP codes can be reused inside their time window	Replay by someone who observes the code within ~60 s	Store last accepted time-step per user
Medium	Single HS256 key without rotation id	Rotating the key logs everyone out; compromise affects all tokens	Use key id (kid) with two active keys, or move to asymmetric signing
Medium	No generic API rate limit in the application	Scraping and abuse of expensive endpoints	WAF rate rules and Redis-backed middleware
Medium	Antivirus scan on uploads not implemented	Infected PDF could reach students	ClamAV job before publish (S-01 T-08)
Low	Free-plan storage is ephemeral	Uploaded files vanish on redeploy	Disk mount or S3

5.10 Appendix · source map

Topic	File
Token and crypto helpers	`backend/app/core/security.py`
OTP, sessions, staff login, TOTP	`backend/app/modules/auth/service.py`
Permission mapping	`backend/app/core/rbac.py`
Auth dependencies	`backend/app/core/deps.py`
CSRF and request context	`backend/app/core/middleware.py`
Settings and guard rails	`backend/app/core/config.py`
Audit chain	`backend/app/modules/audit/service.py`
Payments	`backend/app/modules/payments/service.py`
Access ledger	`backend/app/modules/enrolment/service.py`
Tests engine	`backend/app/modules/qbank/service.py`
Spaced repetition	`backend/app/modules/flashcards/service.py`
API client (CSRF + refresh)	`frontend/src/lib/api.ts`
Proxy and headers	`frontend/next.config.ts`
Deployment	`render.yaml` · `Dockerfile` · `deploy/start.sh`
Design documents	`Documentation/` (A-01…A-08 architecture, S-01…S-03 security, N-01 NFRs, P-xx product)